

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
MÔN: HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU - IS210.Q23
ĐỀ TÀI: HỆ THỐNG QUẢN LÝ KHÁCH SẠN

GVHD: ThS. Đỗ Thị Minh Phụng

Nhóm sinh viên thực hiện:

- | | |
|-----------------------|-----------------------|
| 1. Vũ Thị Phương Linh | MSSV: 20521541 |
| 2. Cao Thái Bảo | MSSV: 24520145 |
| 3. Võ Minh Quân | MSSV: 24521459 |
| 4. Nguyễn Minh Thắng | MSSV: 24521607 |

☞ Thành phố Hồ Chí Minh, tháng 05 năm 2026 ☞

This image shows a full page of white paper with horizontal dotted lines, typical of primary school writing paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Trước tiên, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Đỗ Thị Minh Phụng – giảng viên phụ trách môn Hệ quản trị cơ sở dữ liệu (IS210). Nhờ có những bài giảng tâm huyết, sự hướng dẫn tận tình và những định hướng chuyên môn sắc bén của Cô, chúng em mới có đủ nền tảng kiến thức để hoàn thành đồ án này.

Đồng thời, chúng em cũng xin cảm ơn Khoa Hệ thống thông tin và Trường Đại học Công nghệ Thông tin – ĐHQG-HCM đã tạo điều kiện về cơ sở vật chất, môi trường học tập tốt nhất để chúng em có thể nghiên cứu và phát triển đề tài.

Dù đã nỗ lực hết sức, song do hạn chế về mặt thời gian cũng như kinh nghiệm thực tiễn, đồ án chắc chắn vẫn còn những thiếu sót. Nhóm chúng em rất mong nhận được những nhận xét, đánh giá và góp ý quý báu từ Cô để đồ án được hoàn thiện hơn, đồng thời giúp chúng em tích lũy thêm kinh nghiệm cho những dự án thực tế sau này.

Chúng em xin chân thành cảm ơn!

LỜI MỞ ĐẦU

Trong lĩnh vực kinh doanh dịch vụ lưu trú, việc quản lý thông tin khách hàng, phòng ở và các dịch vụ đi kèm đóng vai trò then chốt. Thay vì thực hiện các phương pháp ghi chép thủ công dễ gây nhầm lẫn, việc áp dụng phần mềm quản lý giúp quy trình vận hành trở nên nhanh chóng và chính xác hơn.

Với mục tiêu vận dụng những kiến thức đã học trong môn "Hệ quản trị cơ sở dữ liệu" (IS210) vào thực tế, nhóm chúng em đã thực hiện đề tài: **"Xây dựng hệ thống Quản lý khách sạn"**.

Đề án này tập trung vào việc thiết kế một cơ sở dữ liệu khoa học để lưu trữ thông tin, đồng thời xây dựng các chức năng nghiệp vụ cần thiết cho một khách sạn như: đặt phòng, quản lý khách hàng, thanh toán và báo cáo doanh thu. Thông qua quá trình thực hiện, nhóm chúng em mong muốn hiểu rõ hơn về cách tổ chức dữ liệu và xử lý các tình huống nghiệp vụ phát sinh trong hệ thống.

Mặc dù đã cố gắng hoàn thiện các chức năng, nhưng do kiến thức và kinh nghiệm còn hạn chế, đề án chắc chắn không tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự góp ý của Cô để có thể học hỏi và cải thiện tốt hơn.

MỤC LỤC

LỜI CẢM ƠN	3
LỜI MỞ ĐẦU	4
MỤC LỤC	5
DANH MỤC BẢNG	9
DANH MỤC HÌNH ẢNH	11
DANH MỤC VIẾT TẮT	12
Chương 1: TỔNG QUAN ĐỀ TÀI	13
1.1Phát biểu bài toán	13
1.2Quy trình nghiệp vụ.....	13
1.2.1 Quy trình đặt phòng	13
1.2.2 Quy trình check-in/check-out.....	14
1.2.3 Quy trình thanh toán	14
1.2.4 Quy trình quản lý phòng và giá.....	14
1.2.5 Quy trình hỗ trợ khách hàng	14
1.3Xác định và phân tích yêu cầu.....	15
1.3.1 Yêu cầu chức năng.....	15
1.3.2 Yêu cầu phi chức năng.....	15
Chương 2: XÁC ĐỊNH VÀ MÔ HÌNH HÓA YÊU CẦU.....	16
2.1Thiết kế mô hình quan hệ.....	16
2.1.1 Lược đồ cơ sở dữ liệu quan hệ.....	16
2.1.2 Mô hình dữ liệu ERD.....	17
2.1.3 Mô hình dữ liệu logic.....	17
2.1.4 Bảng thuyết minh quan hệ và thuộc tính.....	17
2.2Mô tả ràng buộc toàn vẹn.....	28
2.2.1 Ràng buộc toàn vẹn có bối cảnh trên một quan hệ	28
2.2.1.1. Ràng buộc toàn vẹn liên thuộc tính.....	28
2.2.1.2. Ràng buộc toàn vẹn liên bộ	29
2.2.2 Ràng buộc toàn vẹn có bối cảnh trên nhiều quan hệ - Ràng buộc liên bộ - liên quan hệ	30
Chương 3: XÂY DỰNG VÀ QUẢN LÝ GIAO TÁC	32
3.1. Trigger.....	32
3.1.1. Trigger trong Oracle và danh sách các trigger sử dụng	32

3.1.2.	Mô tả chi tiết các trigger	34
3.2.	Stored Procedure	41
3.2.1.	Danh sách các Stored Procedure	41
3.2.2.	Mô tả chi tiết các Stored Procedure	42
3.3	Stored Function	54
3.3.1	Danh sách các Stored Function	54
3.3.2	Mô tả chi tiết các Stored Function	56
Chương 4:	XỬ LÝ TRUY XUẤT ĐỒNG THỜI.....	64
4.1	Các mức cô lập trong Oracle.....	64
4.1.1	Read Committed (mặc định)	64
4.1.2	Serializable.....	65
4.1.3	Repeatable Read.....	65
4.2	Cơ chế khóa (Locking).....	66
4.3	Deadlock.....	66
4.4	Mô tả các hiện tượng đồng thời trong hệ thống	67
4.4.1	Phantom Read – Hai phiên cùng tìm phòng trống	67
4.4.2	Lost Update – Cập nhật trạng thái 2 phòng đồng thời	70
4.4.3	Non-Repeatable Read – Đọc giá phòng thay đổi giữa chừng	74
4.4.4	Deadlock – Hai booking cùng khóa hai phòng theo thứ tự ngược nhau	78
Chương 5:	THIẾT KẾ GIAO DIỆN	84
5.1	Sơ đồ liên kết các màn hình	84
5.2	Danh sách các màn hình.....	84
5.3	Mô tả chi tiết các màn hình	88
5.3.1	Màn hình đăng nhập/đăng ký	88
5.3.1.1.	Giao diện màn hình	88
5.3.1.2.	Mô tả các đối tượng trên màn hình	89
5.3.1.3.	Danh sách biến cố và xử lý tương ứng trên màn hình.....	91
5.3.2	Màn hình trang chủ	93
a.	Giao diện màn hình	93
b.	Mô tả các đối tượng trên màn hình	93
c.	Danh sách biến cố và xử lý tương ứng.....	94

5.3.3. Màn hình Tìm kiếm & Đặt phòng.....	94
a. Giao diện màn hình	94
b. Mô tả các đối tượng trên màn hình	95
c. Danh sách biến cố và xử lý tương ứng.....	96
5.3.4 Màn hình tra cứu đặt phòng	97
a. Giao diện màn hình	97
b. Mô tả các đối tượng trên màn hình	97
c. Danh sách các biến cố và xử lý tương ứng.....	99
5.3.5 Màn hình chat với lễ tân.....	101
a. Giao diện màn hình	101
b. Mô tả các đối tượng trên màn hình	101
c. Danh sách các biến cố và xử lý tương ứng.....	102
5.3.6 Màn hình Dashboard (phía Admin)	103
a. Giao diện màn hình	103
b. Mô tả các đối tượng trên màn hình	103
c. Danh sách các biến cố và xử lý tương ứng.....	104
5.3.7 Màn hình quản lý loại phòng	105
a. Giao diện màn hình	105
b. Mô tả các đối tượng trên màn hình	105
c. Danh sách các biến cố và xử lý tương ứng.....	106
5.3.8 Màn hình quản lý phòng	107
a. Giao diện màn hình	107
b. Mô tả các đối tượng trên màn hình	108
c. Danh sách các biến cố và xử lý tương ứng.....	109
5.3.9 Màn hình quản lý đặt phòng	110
a. Giao diện màn hình	110
b. Mô tả các đối tượng trên màn hình	110
c. Danh sách các biến cố và xử lý tương ứng.....	111

DANH MỤC BẢNG

Bảng 2.1: Bảng AMENITIES.....	14
Bảng 2.2: Bảng CUSTOMERS	14
Bảng 2.3: Bảng ROOM_TYPES	15
Bảng 2.4: Bảng PROMOTIONS	16
Bảng 2.5: Bảng POLICIES.....	16
Bảng 2.6: Bảng USERS.....	17
Bảng 2.7: Bảng ROOM_TYPE_IMAGES	18
Bảng 2.8: Bảng ROOM_TYPE_AMENITIES.....	18
Bảng 2.9: Bảng ROOMS	18
Bảng 2.10: Bảng DAILY_PRICES	19
Bảng 2.11: Bảng SEASONAL_PRICES.....	19
Bảng 2.12: Bảng REVIEWS.....	20
Bảng 2.13: Bảng REVIEW_IMAGES.....	20
Bảng 2.14: Bảng CHAT_CONVERSATIONS	20
Bảng 2.15: Bảng BOOKINGS.....	21
Bảng 2.16: Bảng BOOKING_HISTORY.....	22
Bảng 2.17: Bảng PAYMENT	22
Bảng 2.18: Bảng BOOKING_PROMOTIONS.....	23
Bảng 2.19: Bảng BOOKING_POLICIES	23
Bảng 2.20: Bảng CHAT_MESSAGES.....	23
Bảng 2.21: Bảng FAQs	24
Bảng 2.22: Bảng SALES_REPORTS.....	24
Bảng 2.23: Bảng tầm ảnh hưởng “Ngày check-out phải sau ngày check-in”.....	25
Bảng 2.24: Bảng tầm ảnh hưởng “Ngày kết thúc khuyến mãi phải lớn hơn hoặc bằng ngày bắt đầu”	26
Bảng 2.25: Bảng tầm ảnh hưởng “Số lượt sử dụng mã khuyến mãi không vượt quá giới hạn (max_uses)”	26
Bảng 2.26: Bảng tầm ảnh hưởng “Một booking không áp dụng cùng một mã khuyến mãi hai lần”	26
Bảng 2.27: Bảng tầm ảnh hưởng “Rating đánh giá phải nằm trong khoảng 1.0 đến 5.0”	27

Bảng 2.28: Bảng tầm ảnh hưởng “Trạng thái phòng phải thuộc tập hợp hợp lệ”	27
Bảng 2.29: Bảng tầm ảnh hưởng “Booking chỉ được xác nhận khi thanh toán đạt tối thiểu 50% tổng giá trị”	28
Bảng 3.1: Danh sách các trigger sử dụng trong hệ thống quản lý khách sạn	29
Bảng 3.2: Danh sách các Stored Procedure được sử dụng trong hệ thống quản lý khách sạn	34

DANH MỤC HÌNH ẢNH

DANH MỤC VIẾT TẮT

MVCC

DML

Chương 1: TỔNG QUAN ĐỀ TÀI

1.1 Phát biểu bài toán

Ngành khách sạn tại Việt Nam đang phát triển mạnh mẽ, đặc biệt sau giai đoạn phục hồi du lịch. Tuy nhiên, phần lớn các khách sạn vừa và nhỏ vẫn đang quản lý hoạt động theo phương thức thủ công hoặc sử dụng các phần mềm rời rạc, thiếu kết nối. Điều này dẫn đến nhiều bất cập: thông tin đặt phòng dễ bị trùng lặp, sai sót, quy trình check-in/check-out kéo dài, báo cáo doanh thu thiếu chính xác và việc chăm sóc khách hàng thiếu nhất quán.

Cụ thể, nhân viên lễ tân thường xuyên phải tra cứu phòng trống bằng sổ sách hoặc bảng tính, dễ gây nhầm lẫn trong các mùa cao điểm. Chính sách giá linh hoạt (giá ngày lễ, giá theo mùa) khó áp dụng thủ công. Mã khuyến mãi không được kiểm soát chặt chẽ dẫn đến lạm dụng. Quản lý không có số liệu tổng hợp kịp thời để ra quyết định kinh doanh. Đặc biệt, khách hàng không có kênh tra cứu, đặt phòng hay hỗ trợ trực tuyến, làm giảm trải nghiệm và tính cạnh tranh của khách sạn.

Để giải quyết các vấn đề trên, nhóm phát triển đã xây dựng Hệ thống Quản lý Khách sạn – một nền tảng toàn diện tích hợp trên ứng dụng desktop, cung cấp đầy đủ các chức năng quản lý đặt phòng, khách hàng, phòng, giá động, thanh toán, báo cáo doanh thu và giao tiếp thời gian thực qua WebSocket. Hệ thống hướng tới tối ưu hóa toàn bộ quy trình vận hành khách sạn, giảm thiểu sai sót thủ công và nâng cao chất lượng dịch vụ khách hàng.

1.2 Quy trình nghiệp vụ

Hệ thống hotel CRM bao gồm 5 quy trình nghiệp vụ chính:

1.2.1 Quy trình đặt phòng

Khách hàng truy cập hệ thống và tìm kiếm phòng trống bằng cách nhập khoảng thời gian check-in, check-out mong muốn và loại phòng. Hệ thống tra cứu cơ sở dữ liệu và trả về danh sách các phòng còn trống kèm thông tin giá theo ngày, tiện nghi và ảnh minh họa. Khách hàng chọn phòng phù hợp, điền thông tin cá nhân (nếu chưa có tài khoản) và nhập mã khuyến mãi nếu có.

Sau khi khách hàng xác nhận, hệ thống tự động tính tổng giá trị booking dựa trên công thức đa tầng: cộng phụ phí ngày đặc biệt (daily price) và nhân với hệ số mùa

(seasonal multiplier) tương ứng cho từng đêm trong kỳ đặt phòng, sau đó trừ đi giá trị khuyến mãi. Booking được tạo với trạng thái PENDING và chờ xác nhận thanh toán.

1.2.2 Quy trình check-in/check-out

Khi khách đến nhận phòng, nhân viên lễ tân tra cứu booking theo mã xác nhận hoặc thông tin khách hàng. Hệ thống hiển thị đầy đủ chi tiết đặt phòng và lịch sử thanh toán. Lễ tân xác nhận check-in, cập nhật trạng thái booking.

Khi khách trả phòng, lễ tân thực hiện thao tác check-out, hệ thống cập nhật trạng thái booking, phòng sẽ được chuyển sang trạng thái chờ dọn dẹp. Toàn bộ lịch sử đều được ghi lại vào bảng booking_history.

1.2.3 Quy trình thanh toán

Khách hàng có thể thanh toán trước hoặc tại quầy. Hệ thống hỗ trợ thanh toán nhiều lần cho một booking. Khi tổng số tiền đã thanh toán đạt ít nhất 50% giá trị booking, trigger sẽ tự động chuyển trạng thái booking từ PENDING sang CONFIRMED và ghi nhận vào booking_history. Nhân viên lễ tân hoặc khách hàng có thể xuất hóa đơn PDF ngay sau khi thanh toán thành công.

1.2.4 Quy trình quản lý phòng và giá

Hệ thống hỗ trợ thiết lập giá đặc biệt theo ngày cho các dịp lễ lớn như 30/4, 2/9, Giáng sinh, Giao thừa và giá theo mùa (seasonal prices) với hệ số nhân và độ ưu tiên. Khi có nhiều mức giá theo mùa áp dụng cùng lúc, hệ thống tự động chọn mức có giá trị cao nhất. Ngoài ra, người quản lý có thể tạo mã khuyến mãi dạng phần trăm hoặc số tiền cố định với giới hạn thời gian và số lượt sử dụng.

1.2.5 Quy trình hỗ trợ khách hàng

Khách hàng có thể giao tiếp trực tiếp với nhân viên lễ tân qua tính năng chat tích hợp WebSocket STOMP theo thời gian thực. Mỗi phiên hội thoại được lưu đầy đủ lịch sử tin nhắn với trạng thái đọc/chưa đọc. Ngoài ra, hệ thống cung cấp trang FAQ giúp khách hàng tự giải đáp các thắc mắc phổ biến. Khách hàng đã từng lưu trú cũng có thể để lại đánh giá sau khi check-out; đánh giá sẽ qua quy trình duyệt của quản lý trước khi hiển thị công khai.

1.3 Xác định và phân tích yêu cầu

1.3.1 Yêu cầu chức năng

- Yêu cầu lưu trữ: Hệ thống phải tổ chức lưu trữ đầy đủ, nhất quán các nhóm dữ liệu bao gồm: thông tin tài khoản người dùng, hồ sơ khách hàng, danh sách 8 loại phòng và 40 phòng cụ thể, tiện nghi, hình ảnh, các bảng giá (giá ngày, giá mùa), thông tin đặt phòng, lịch sử đặt phòng, khuyến mãi, chính sách, thanh toán, đánh giá và lịch sử tin nhắn.
- Yêu cầu tính toán:
 - Hệ thống phải tự động tính toán tổng chi phí đặt phòng dựa trên sự kết hợp phức tạp của giá cơ bản, giá theo ngày đặc biệt và hệ số giá theo mùa.
 - Thực hiện phép trừ chiết khấu dựa trên các mã khuyến mãi (theo tỉ lệ phần trăm hoặc số tiền cố định).
 - Tính toán, tổng hợp báo cáo doanh thu định kỳ của khách sạn.
- Yêu cầu tra cứu: Hệ thống cung cấp công cụ cho khách hàng tra cứu phòng trống theo nhu cầu; cho phép nhân viên và quản trị viên tra cứu chi tiết lịch sử đặt phòng, tình trạng thanh toán, thông tin khách hàng và lịch sử thay đổi của các hóa đơn.
- Yêu cầu kết xuất và thống kê: Hệ thống hỗ trợ lập báo cáo doanh thu thống kê theo các tiêu chí thời gian. Có khả năng xuất các hóa đơn thanh toán ra định dạng file PDF hỗ trợ tiếng Việt hoàn chỉnh.

1.3.2 Yêu cầu phi chức năng

- Yêu cầu về hiệu năng, bảo mật và khả năng mở rộng: Kiến trúc 3 tầng (Frontend, Backend, Database) phải hoạt động trơn tru, cho phép hệ thống đáp ứng tốt các truy vấn đồng thời và dễ dàng mở rộng nghiệp vụ trong tương lai. Việc ứng dụng Docker để container hóa cơ sở dữ liệu giúp việc triển khai và tái tạo môi trường nhanh chóng, ổn định, đồng thời phải bảo đảm an toàn thông tin cơ sở dữ liệu trên Oracle 19c. Ứng dụng chặt chẽ các trigger để quản lý logic nghiệp vụ ở tầng cơ sở dữ liệu, duy trì ràng buộc toàn vẹn và chống thất thoát dữ liệu.
- Yêu cầu về giao tiếp thời gian thực: Tính năng nhắn tin hỗ trợ khách hàng yêu cầu kết nối liên tục, độ trễ thấp thông qua giao thức WebSocket (STOMP) liên kết giữa Client và Server, giúp đảm bảo tương tác thông suốt giữa khách hàng và Admin.

Chương 2: XÁC ĐỊNH VÀ MÔ HÌNH HÓA YÊU CẦU

2.1 Thiết kế mô hình quan hệ

2.1.1 Lược đồ cơ sở dữ liệu quan hệ

AMENITIES(id, name, icon, category, description)

CUSTOMERS(id, first_name, last_name, email, phone, id_number, nationality, date_of_birth, address, notes, is_vip, created_at, updated_at)

ROOM_TYPES(id, name, description, capacity, base_price, created_at, updated_at)

PROMOTIONS(id, code, description, discount_type, discount_value, start_date, end_date, min_nights, max_uses, used_count, created_at, updated_at)

POLICIES(id, type, title, content, language, version, updated_at)

USERS(id, first_name, last_name, username, email, phone_number, password, role, status, created_at, updated_at)

ROOM_TYPE_IMAGES(room_type_id, image_url)

ROOM_TYPE_AMENITIES(room_type_id, amenity_id)

ROOMS(id, room_type_id, room_number, floor, status, notes, created_at, updated_at)

DAILY_PRICES(id, room_type_id, price_date, price, reason)

SEASONAL_PRICES(id, room_type_id, name, start_date, end_date, price_multiplier, priority)

REVIEWS(id, user_id, rating, content, status, created_at)

REVIEW_IMAGES(review_id, image_url)

CHAT_CONVERSATIONS(id, session_id, user_id, status, created_at, updated_at)

BOOKINGS(id, customer_id, room_id, check_in_date, check_out_date, total_price, raw_price, discount_amount, status, special_requests, created_at)

BOOKING_HISTORY(id, booking_id, action, performed_by, timestamp, notes)

PAYMENT(id, booking_id, payment_date, amount, status)

BOOKING_PROMOTIONS(booking_id, promotion_id)

BOOKING_POLICIES(booking_id, policy_id)

CHAT_MESSAGES(id, chat_conversation_id, admin_id, sender_type, content, is_read, read_at, created_at)

FAQS(id, question, answer, is_active, created_at, updated_at)

SALES_REPORTS(id, report_date, total_bookings, gross_revenue, discount_amount, net_revenue, room_occupancy_rate, created_at)

2.1.2 Mô hình dữ liệu ERD

2.1.3 Mô hình dữ liệu logic

2.1.4 Bảng thuyết minh quan hệ và thuộc tính

a. Bảng AMENITIES

Bảng AMENITIES lưu thông tin các tiện ích của khách sạn.

Bảng 2.1: Bảng AMENITIES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã tiện ích	PRIMARY KEY

name	VARCHAR2(100)	Tên tiện ích	NOT NULL
icon	VARCHAR2(50)	Biểu tượng	
category	VARCHAR2(50)	Nhóm tiện ích	
description	VARCHAR2(500)	Mô tả tiện ích	

b. Bảng CUSTOMERS

Bảng CUSTOMERS lưu thông tin khách hàng.

Bảng 2.2: Bảng CUSTOMERS

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã khách hàng	PRIMARY KEY
first_name	VARCHAR2(100)	Họ khách hàng	NOT NULL
last_name	VARCHAR2(100)	Tên khách hàng	NOT NULL
email	VARCHAR2(255)	Email khách hàng	NOT NULL, UNIQUE
phone	VARCHAR2(20)	Số điện thoại	
id_number	VARCHAR2(50)	CCCD/Hộ chiếu	
nationality	VARCHAR2(50)	Quốc tịch	DEFAULT 'Việt Nam'
date_of_birth	DATE	Ngày sinh	
address	VARCHAR2(500)	Địa chỉ	
notes	VARCHAR2(1000)	Ghi chú	
is_vip	NUMBER(1)	Trạng thái VIP	NOT NULL
created_at	TIMESTAMP	Ngày tạo	NOT NULL
updated_at	TIMESTAMP	Ngày cập nhật	NOT NULL

c. Bảng ROOM_TYPES

Bảng ROOM_TYPES lưu thông tin các loại phòng.

Bảng 2.3: Bảng ROOM_TYPES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã loại phòng	PRIMARY KEY
name	VARCHAR2(100)	Tên loại phòng	NOT NULL
description	VARCHAR2(2000)	Mô tả loại phòng	
capacity	NUMBER(10,0)	Sức chứa tối đa	NOT NULL
base_price	NUMBER(12,2)	Giá cơ bản	NOT NULL
created_at	TIMESTAMP	Ngày tạo	NOT NULL
updated_at	TIMESTAMP	Ngày cập nhật	NOT NULL

d. Bảng PROMOTIONS

Bảng PROMOTIONS lưu thông tin khuyến mãi.

Bảng 2.4: Bảng PROMOTIONS

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã khuyến mãi	PRIMARY KEY
code	VARCHAR2(50)	Mã giảm giá	NOT NULL, UNIQUE
description	VARCHAR2(500)	Mô tả	
discount_type	VARCHAR2(20)	Loại giảm giá	NOT NULL
discount_value	NUMBER(12,2)	Giá trị giảm	NOT NULL
start_date	DATE	Ngày bắt đầu	NOT NULL
end_date	DATE	Ngày hết hạn	NOT NULL
min_nights	NUMBER(10,0)	Số đêm tối thiểu	
max_uses	NUMBER(10,0)	Số lượt tối đa	

used_count	NUMBER(10,0)	Số lượt đã dùng	DEFAULT 0
created_at	TIMESTAMP	Ngày tạo	NOT NULL
updated_at	TIMESTAMP	Ngày cập nhật	NOT NULL

e. Bảng POLICIES

Bảng POLICIES lưu các chính sách khách sạn.

Bảng 2.5: Bảng POLICIES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã chính sách	PRIMARY KEY
type	VARCHAR2(50)	Loại chính sách	NOT NULL
titles	VARCHAR2(200)	Tiêu đề	NOT NULL
content	CLOB	Nội dung	
language	VARCHAR2(10)	Ngôn ngữ	
version	NUMBER(10,0)	Phiên bản	NOT NULL
updated_at	TIMESTAMP	Ngày cập nhật	NOT NULL

f. Bảng USERS

Bảng USERS lưu thông tin tài khoản người dùng.

Bảng 2.6: Bảng USERS

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã người dùng	PRIMARY KEY
first_name	VARCHAR2(100)	Họ	NOT NULL
last_name	VARCHAR2(100)	Tên	NOT NULL
username	VARCHAR2(100)	Tên đăng nhập	NOT NULL, UNIQUE

email	VARCHAR2(255)	Email khách hàng	NOT NULL, UNIQUE
phone_number	VARCHAR2(20)	Số điện thoại	
password	VARCHAR2(255)	Mật khẩu	NOT NULL
role	VARCHAR2(20)	Vai trò	NOT NULL
status	VARCHAR2(20)	Trạng thái	NOT NULL
created_at	TIMESTAMP	Ngày tạo	NOT NULL
updated_at	TIMESTAMP	Ngày cập nhật	NOT NULL

g. Bảng ROOM_TYPE_IMAGES

Bảng ROOM_TYPE_IMAGES lưu hình ảnh của loại phòng.

Bảng 2.7: Bảng ROOM_TYPE_IMAGES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
room_type_id	NUMBER	Mã loại phòng	PRIMARY KEY, FOREIGN KEY
image_url	VARCHAR2(500)	URL đến ảnh	PRIMARY KEY, NOT NULL

h. Bảng ROOM_TYPE_AMENITIES

Bảng ROOM_TYPE_AMENITIES quản lý tiện ích của từng loại phòng.

Bảng 2.8: Bảng ROOM_TYPE_AMENITIES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
room_type_id	NUMBER	Mã loại phòng	PRIMARY KEY, FOREIGN KEY

amenity_id	NUMBER	Mã tiện ích	PRIMARY KEY, FOREIGN KEY
------------	--------	-------------	-----------------------------

i. Bảng ROOMS

Bảng ROOMS lưu thông tin từng phòng.

Bảng 2.9: Bảng ROOMS

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã phòng	PRIMARY KEY
room_type_id	NUMBER	Loại phòng	FOREIGN KEY, NOT NULL
room_number	VARCHAR2(20)	Số phòng	NOT NULL
floor	NUMBER(10,0)	Tầng	
status	VARCHAR2(20)	Trạng thái phòng	NOT NULL
notes	VARCHAR2(500)	Ghi chú	
created_at	TIMESTAMP	Ngày tạo	NOT NULL
updated_at	TIMESTAMP	Ngày cập nhật	NOT NULL

j. Bảng DAILY_PRICES

Bảng DAILY_PRICES lưu giá phòng theo ngày.

Bảng 2.10: Bảng DAILY_PRICES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã giá ngày	PRIMARY KEY
room_type_id	NUMBER	Loại phòng	FOREIGN KEY, NOT NULL
price_date	DATE	Ngày áp dụng	NOT NULL
price	NUMBER(12,2)	Giá phòng	NOT NULL

reason	VARCHAR2(500)	Lý do thay đổi	
--------	---------------	----------------	--

k. Bảng SEASONAL_PRICES

Bảng SEASONAL_PRICES lưu hệ số giá theo mùa.

Bảng 2.11: Bảng SEASONAL_PRICES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã giá theo mùa	PRIMARY KEY
room_type_id	NUMBER	Loại phòng	FOREIGN KEY, NOT NULL
name	VARCHAR2(100)	Tên mùa	NOT NULL
start_date	DATE	Ngày bắt đầu	NOT NULL
end_date	DATE	Ngày kết thúc	NOT NULL
price_multiplier	NUMBER(10,2)	Hệ số giá	NOT NULL
priority	NUMBER(10,0)	Độ ưu tiên	NOT NULL

l. Bảng REVIEWS

Bảng REVIEWS lưu đánh giá của khách hàng.

Bảng 2.12: Bảng REVIEWS

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã đánh giá	PRIMARY KEY
user_id	NUMBER	Người đánh giá	FOREIGN KEY, NOT NULL
rating	NUMBER(2,1)	Điểm đánh giá	NOT NULL
content	CLOB	Nội dung đánh giá	
status	VARCHAR2(20)	Trạng thái duyệt	
created_at	TIMESTAMP	Ngày tạo	NOT NULL

m. Bảng REVIEW_IMAGES

Bảng REVIEW_IMAGES lưu hình ảnh đánh giá.

Bảng 2.13: Bảng REVIEW_IMAGES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
review_id	NUMBER	Mã đánh giá	PRIMARY KEY, FOREIGN KEY
image_url	VARCHAR2(500)	URL dẫn đến ảnh	PRIMARY KEY, NOT NULL

n. Bảng CHAT_CONVERSATIONS

Bảng CHAT_CONVERSATIONS quản lý các cuộc hội thoại.

Bảng 2.14: Bảng CHAT_CONVERSATIONS

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã cuộc hội thoại	PRIMARY KEY
session_id	VARCHAR2(100)	Session chat	NOT NULL, UNIQUE
user_id	NUMBER	Người dùng	FOREIGN KEY
status	VARCHAR2(20)	Trạng thái	NOT NULL
created_at	TIMESTAMP	Ngày tạo	NOT NULL
updated_at	TIMESTAMP	Ngày cập nhật	NOT NULL

o. Bảng BOOKINGS

Bảng BOOKINGS lưu thông tin đặt phòng.

Bảng 2.15: Bảng BOOKINGS

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
------------	--------------	---------	-----------

id	NUMBER	Mã booking	PRIMARY KEY
customer_id	NUMBER	Mã khách hàng	FOREIGN KEY, NOT NULL
room_id	NUMBER	Phòng	FOREIGN KEY, NOT NULL
check_in_date	DATE	Ngày nhận phòng	NOT NULL
check_out_date	DATE	Ngày trả phòng	NOT NULL
total_price	NUMBER(12,2)	Tổng tiền	NOT NULL
raw_price	NUMBER(12,2)	Giá gốc	NOT NULL
discount_amount	NUMBER(12,2)	Giảm giá	NOT NULL
status	VARCHAR2(20)	Trạng thái booking	NOT NULL
special_requests	VARCHAR2(1000)	Yêu cầu đặc biệt	
created_at	TIMESTAMP	Ngày tạo	NOT NULL

p. Bảng BOOKING_HISTORY

Bảng BOOKING_HISTORY lưu lịch sử thay đổi booking.

Bảng 2.16: Bảng BOOKING_HISTORY

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã lịch sử	PRIMARY KEY
booking_id	NUMBER	Booking liên quan	FOREIGN KEY, NOT NULL
action	VARCHAR2(50)	Hành động	NOT NULL
performed_by	VARCHAR2(100)	Người thực hiện	

timestamp	TIMESTAMP	Thời gian thực hiện	NOT NULL
notes	VARCHAR2(500)	Ghi chú	

q. Bảng PAYMENT

Bảng PAYMENT lưu thông tin thanh toán.

Bảng 2.17: Bảng PAYMENT

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã thanh toán	PRIMARY KEY
booking_id	NUMBER	Booking liên quan	FOREIGN KEY
payment_date	TIMESTAMP	Ngày thanh toán	NOT NULL
amount	NUMBER(12,2)	Số tiền	NOT NULL
status	VARCHAR2(20)	Trạng thái	NOT NULL

r. Bảng BOOKING_PROMOTIONS

Bảng BOOKING_PROMOTIONS quản lý khuyến mãi áp dụng cho booking.

Bảng 2.18: Bảng BOOKING_PROMOTIONS

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
booking_id	NUMBER	Booking	PRIMARY KEY, FOREIGN KEY
promotion_id	NUMBER	Khuyến mãi	PRIMARY KEY, FOREIGN KEY

s. Bảng BOOKING_POLICIES

Bảng BOOKING_POLICIES quản lý chính sách áp dụng cho booking.

Bảng 2.19: Bảng BOOKING_POLICIES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
booking_id	NUMBER	Booking	PRIMARY KEY, FOREIGN KEY
policy_id	NUMBER	Chính sách	PRIMARY KEY, FOREIGN KEY

t. Bảng CHAT_MESSAGES

Bảng CHAT_MESSAGES lưu tin nhắn chat.

Bảng 2.20: Bảng CHAT_MESSAGES

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã tin nhắn	PRIMARY KEY
chat_conversation_id	NUMBER	Cuộc hội thoại	FOREIGN KEY, NOT NULL
admin_id	NUMBER	Admin gửi	FOREIGN KEY
sender_type	VARCHAR2(20)	Loại người gửi	NOT NULL
content	CLOB	Nội dung tin nhắn	NOT NULL
is_read	NUMBER(1)	Trạng thái đã đọc	NOT NULL
read_at	TIMESTAMP	Thời gian đọc	
created_at	TIMESTAMP	Ngày gửi	NOT NULL

u. Bảng FAQs

Bảng FAQs lưu các câu hỏi thường gặp.

Bảng 2.21: Bảng FAQs

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã FAQs	PRIMARY KEY

question	VARCHAR2(500)	Câu hỏi	NOT NULL
answer	CLOB	Câu trả lời	
is_active	NUMBER(1)	Trạng thái hoạt động	
created_at	TIMESTAMP	Ngày tạo	NOT NULL
updated_at	TIMESTAMP	Ngày cập nhật	NOT NULL

v. Bảng SALES_REPORTS

Bảng SALES_REPORTS lưu báo cáo doanh thu.

Bảng 2.22: Bảng SALES_REPORTS

Thuộc tính	Kiểu dữ liệu	Ý nghĩa	Ràng buộc
id	NUMBER	Mã báo cáo	PRIMARY KEY
report_date	DATE	Ngày báo cáo	NOT NULL, UNIQUE
total_bookings	NUMBER(10,0)	Tổng booking	
gross_revenue	NUMBER(15,2)	Doanh thu gộp	
discount_amount	NUMBER(15,2)	Tổng giảm giá	
net_revenue	NUMBER(15,2)	Doanh thu thực	NOT NULL
room_occupancy_rate	NUMBER(5,2)	Tỷ lệ lấp đầy	
created_at	TIMESTAMP	Ngày tạo	NOT NULL

2.2 Mô tả ràng buộc toàn vẹn

2.2.1 Ràng buộc toàn vẹn có bối cảnh trên một quan hệ

2.2.1.1. Ràng buộc toàn vẹn liên thuộc tính

- a. Ngày check-out phải sau ngày check-in

- Bối cảnh: BOOKINGS
- Nội dung:

$$\forall bk \in BOOKINGS: bk.check_out_date > bk.check_in_date$$

- Bảng tầm ảnh hưởng:

Bảng 2.23: Bảng tầm ảnh hưởng “Ngày check-out phải sau ngày check-in”

R1	Thêm	Xóa	Sửa
BOOKINGS	+		+(check_in_date, check_out_date)

b. Ngày kết thúc khuyến mãi phải lớn hơn hoặc bằng ngày bắt đầu

- Bối cảnh: PROMOTIONS
- Nội dung:

$$\forall p \in PROMOTIONS: p.end_date \geq p.start_date$$

- Bảng tầm ảnh hưởng:

Bảng 2.24: Bảng tầm ảnh hưởng “Ngày kết thúc khuyến mãi phải lớn hơn hoặc bằng ngày bắt đầu”

R1	Thêm	Xóa	Sửa
PROMOTIONS	+		+(end_date, start_date)

2.2.1.2. Ràng buộc toàn vẹn liên bộ

a. Một booking không áp dụng cùng một mã khuyến mãi hai lần

- Bối cảnh: BOOKING_PROMOTIONS
- Nội dung:

$$\forall bp1, bp2 \in BOOKING_PROMOTIONS: bp1 \neq bp2 \Rightarrow (bp1.booking_id \neq bp2.booking_id \cup bp1.promotion_id \neq bp2.promotion_id)$$

- Bảng tầm ảnh hưởng:

Bảng 2.26: Bảng tầm ảnh hưởng “Một booking không áp dụng cùng một mã khuyến mãi hai lần”

R1	Thêm	Xóa	Sửa
BOOKING_PROMOTIONS	+		+(booking_id, promotion_id)

b. Rating đánh giá phải nằm trong khoảng 1.0 đến 5.0

- Bối cảnh: REVIEWS
- Nội dung:

$$\forall r \in REVIEWS: 1 \leq r.rating \leq 5$$

- Bảng tầm ảnh hưởng:

Bảng 2.27: Bảng tầm ảnh hưởng “Rating đánh giá phải nằm trong khoảng 1.0 đến 5.0”

R1	Thêm	Xóa	Sửa
REVIEWS	+		+(rating)

c. Trạng thái phòng phải thuộc tập hợp hợp lệ

- Bối cảnh: ROOMS
- Nội dung:

$$\forall rm \in ROOMS: rm.status$$

$$\in \{ 'AVAILABLE', 'OCCUPIED', 'MAINTENANCE', 'RESERVED' \}$$

- Bảng tầm ảnh hưởng:

Bảng 2.28: Bảng tầm ảnh hưởng “Trạng thái phòng phải thuộc tập hợp hợp lệ”

R1	Thêm	Xóa	Sửa
ROOMS	+		+(status)

2.2.2 Ràng buộc toàn vẹn có bối cảnh trên nhiều quan hệ - Ràng buộc liên bộ - liên quan hệ

a. Số lượt sử dụng mã khuyến mãi không vượt quá giới hạn (max_uses)

- Bối cảnh: PROMOTIONS, BOOKING_PROMOTIONS
- Nội dung:

$\forall p \in PROMOTIONS: p.max_uses \text{ IS NULL } \cup p.used_count \leq p.max_uses$

- Bảng tầm ảnh hưởng:

Bảng 2.25: Bảng tầm ảnh hưởng “Số lượt sử dụng mã khuyến mãi không vượt quá giới hạn (*max_uses*)”

R1	Thêm	Xóa	Sửa
PROMOTIONS			+ (<i>max_uses</i>)
BOOKING_PROMOTIONS	+		+ (<i>promotion_id</i>)

b. Booking chỉ được xác nhận khi thanh toán đạt tối thiểu 50% tổng giá trị

- Bối cảnh: BOOKINGS, PAYMENT
- Nội dung:

$\forall bk \in BOOKINGS: bk.status = 'CONFIRMED'$

$$\Rightarrow \sum_{p \in PAYMENT, p.booking_id = bk.id, p.status = 'SUCCESS'} p.amount \geq \frac{bk.total_price}{2}$$

- Bảng tầm ảnh hưởng:

Bảng 2.29: Bảng tầm ảnh hưởng “Booking chỉ được xác nhận khi thanh toán đạt tối thiểu 50% tổng giá trị”

R1	Thêm	Xóa	Sửa
BOOKING	+		+ (<i>status</i> , <i>total_price</i>)
PAYMENT	+		+ (<i>amount</i> , <i>status</i> , <i>booking_id</i>)

Chương 3: XÂY DỰNG VÀ QUẢN LÝ GIAO TÁC

3.1. Trigger

3.1.1. Trigger trong Oracle và danh sách các trigger sử dụng

Trigger là một chương trình lưu trữ sẽ tự động thực thi khi một sự kiện xác định xảy ra trên cơ sở dữ liệu. Trigger được viết để phản hồi lại các sự kiện phát sinh khi thực hiện các câu lệnh DML (INSERT, UPDATE, DELETE), DDL (CREATE, ALTER, DROP), hoặc các sự kiện cấp cơ sở dữ liệu (LOGON, LOGOFF, STARTUP, SHUTDOWN).

Lợi ích khi sử dụng trigger:

- Tự động thực thi logic nghiệp vụ mà không cần can thiệp thủ công.
- Đảm bảo tính toàn vẹn dữ liệu và nhất quán giữa các bảng liên quan.
- Ngăn ngừa các giao tác không hợp lệ trước khi dữ liệu được ghi xuống.
- Tự động ghi nhật ký (audit log) các thay đổi trên hệ thống.

Cú pháp tổng quát:

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF}
{INSERT [OR] | UPDATE [OR] | DELETE}
[OF col_name]
ON table_name
[FOR EACH ROW]
[WHEN (condition)]
DECLARE
    -- Khai báo biến
BEGIN
    -- Các câu lệnh thực thi
EXCEPTION
    -- Xử lý ngoại lệ
END;
```

Giải thích các thành phần:

- **CREATE [OR REPLACE] TRIGGER trigger_name:** Tạo mới hoặc thay thế trigger đã tồn tại với tên trigger_name.

- **{BEFORE | AFTER | INSTEAD OF}**: Xác định thời điểm trigger được kích hoạt — trước, sau khi câu lệnh DML thực thi, hoặc thay thế thao tác trên view.
- **{INSERT | UPDATE | DELETE}**: Chỉ định sự kiện DML nào kích hoạt trigger.
- **[OF col_name]**: Giới hạn trigger chỉ kích hoạt khi cột chỉ định bị cập nhật (dùng với UPDATE).
- **ON table_name**: Bảng mà trigger theo dõi.
- **[FOR EACH ROW]**: Khi có mệnh đề này, trigger thực thi một lần cho mỗi dòng bị ảnh hưởng (row-level trigger). Nếu không có, trigger thực thi một lần cho toàn bộ câu lệnh (statement-level trigger).
- **:NEW / :OLD**: Biến đặc biệt tham chiếu đến giá trị mới và cũ của dòng dữ liệu đang được xử lý.

Ngoài dạng trigger thông thường, Oracle còn hỗ trợ COMPOUND TRIGGER — cho phép định nghĩa nhiều thời điểm kích hoạt (BEFORE STATEMENT, BEFORE EACH ROW, AFTER EACH ROW, AFTER STATEMENT) trong cùng một trigger. Đây là giải pháp tránh lỗi mutating table khi trigger cần đọc chính bảng mà nó đang theo dõi.

Dưới đây là danh sách các trigger được sử dụng trong hệ thống quản lý khách sạn:

Bảng 3.1: Danh sách các trigger sử dụng trong hệ thống quản lý khách sạn

STT	Tên trigger	Thao tác	Bảng	Nội dung
1	trg_booking_room_status	INSERT OR UPDATE	BOOKINGS	Tự động cập nhật trạng thái phòng theo booking

2	trg_bpromo_biu	INSERT OR UPDATE	BOOKING_PROMOTIONS	Quản lý số lượt dùng mã khuyến mãi
3	trg_payment_confirm_booking	INSERT OR UPDATE	PAYMENT	Xác nhận booking khi thanh toán đủ 50%
4	trg_total_price	INSERT OR UPDATE	BOOKINGS	Công thức tính giá đa tầng

3.1.2. Mô tả chi tiết các trigger

a. trg_booking_room_status

- Tên trigger: trg_booking_room_status
- Thao tác: INSERT OR UPDATE
- Bối cảnh: BOOKINGS, ROOMS
- Chức năng: Mỗi khi trạng thái của một đơn đặt phòng thay đổi, trigger tự động cập nhật trạng thái tương ứng của phòng liên kết, đảm bảo tính nhất quán giữa hai bảng mà không cần lập trình viên phải gọi thêm câu lệnh UPDATE thủ công.

Các bước thực hiện:

- [1] Nhận giá trị trạng thái mới của booking (:NEW.status).
- [2] Xác định trạng thái phòng tương ứng theo quy tắc ánh xạ:

- [2.1] Nếu trạng thái booking là 'CONFIRMED' → cập nhật phòng thành 'RESERVED'.
- [2.2] Nếu trạng thái booking là 'CHECKED_IN' → cập nhật phòng thành 'OCCUPIED'.
- [2.3] Nếu trạng thái booking là 'CHECKED_OUT' → cập nhật phòng thành 'MAINTENANCE'.
- [2.4] Các trạng thái khác ('PENDING', 'CANCELLED') → cập nhật phòng thành 'AVAILABLE'.
- [3] Thực hiện cập nhật bảng rooms với trạng thái vừa xác định cho phòng có id = :NEW.room_id.

Mã PL/SQL:

```
CREATE OR REPLACE TRIGGER trg_booking_room_status
AFTER INSERT OR UPDATE OF status ON bookings
FOR EACH ROW
BEGIN
    UPDATE rooms
    SET status =
        CASE :NEW.status
            WHEN 'CONFIRMED' THEN 'RESERVED'
            WHEN 'CHECKED_IN' THEN 'OCCUPIED'
            WHEN 'CHECKED_OUT' THEN 'MAINTENANCE'
            ELSE 'AVAILABLE'
        END
    WHERE :NEW.room_id = rooms.room_id;
```

b. trg_bpromo_biu

- Tên trigger: trg_bpromo_biu
- Thao tác: INSERT OR UPDATE
- Bối cảnh: BOOKING_PROMOTIONS, PROMOTIONS
- Mô tả: Trigger đảm bảo ràng buộc toàn vẹn "Số lượt sử dụng mã khuyến mãi không vượt quá giới hạn (max_uses)". Khi một booking được liên kết với mã khuyến mãi, trigger kiểm tra giới hạn và tự động tăng used_count. Khi booking bị hủy dẫn đến xóa liên kết, used_count tự động giảm. Sử dụng SELECT ... FOR UPDATE để khóa dòng, tránh race condition khi nhiều người dùng áp dụng cùng một mã đồng thời.

Các bước thực hiện (trường hợp INSERT):

- [1] Đọc used_count và max_uses của mã khuyến mãi từ bảng promotions, đồng thời khóa dòng (FOR UPDATE) để tránh xung đột đồng thời.
- [2] Kiểm tra giới hạn sử dụng:
 - [2.1] Nếu max_uses không NULL và used_count $\geq$ max_uses $\rightarrow$ báo lỗi -20001 và hủy giao tác.
 - [2.2] Nếu còn lượt $\rightarrow$ tăng used_count thêm 1.
- [3] Cho phép giao tác.

Các bước thực hiện (trường hợp UPDATE):

- [1] Nếu promotion_id không thay đổi $\rightarrow$ không làm gì.
- [2] Nếu promotion_id thay đổi:
 - [2.1] Giảm used_count của mã cũ (OLD.promotion_id) đi 1.
 - [2.2] Kiểm tra giới hạn của mã mới (NEW.promotion_id). Nếu vượt quá $\rightarrow$ báo lỗi -20002 và hủy giao tác.
 - [2.3] Nếu hợp lệ $\rightarrow$ tăng used_count của mã mới thêm 1.
- [3] Cho phép giao tác.

Mã PL/SQL:

```
CREATE OR REPLACE TRIGGER trg_bpromo_biu
BEFORE INSERT OR UPDATE ON booking_promotions
FOR EACH ROW
DECLARE
    v_used_count NUMBER;
    v_max_uses   NUMBER;
BEGIN
    IF INSERTING THEN
        SELECT used_count, max_uses
        INTO v_used_count, v_max_uses
        FROM promotions
        WHERE id = :NEW.promotion_id
```

```

FOR UPDATE;

IF v_max_uses IS NOT NULL AND v_used_count >= v_max_uses THEN
    RAISE_APPLICATION_ERROR(-20001, 'Promotion has reached max usage');
END IF;

UPDATE promotions
SET used_count = used_count + 1
WHERE id = :NEW.promotion_id;
END IF;

IF UPDATING THEN
    IF :OLD.promotion_id != :NEW.promotion_id THEN
        UPDATE promotions
        SET used_count = used_count - 1
        WHERE id = :OLD.promotion_id;

        SELECT used_count, max_uses
        INTO v_used_count, v_max_uses
        FROM promotions
        WHERE id = :NEW.promotion_id
        FOR UPDATE;

        IF v_max_uses IS NOT NULL AND v_used_count >= v_max_uses THEN
            RAISE_APPLICATION_ERROR(-20002, 'New promotion exceeded max
usage');
        END IF;

        UPDATE promotions
        SET used_count = used_count + 1
        WHERE id = :NEW.promotion_id;
    END IF;
END IF;
END;

```

c. trg_payment_confirm_booking

- Tên trigger: trg_payment_confirm_booking

- Thao tác: INSERT OR UPDATE (COMPOUND TRIGGER)
- Bối cảnh: PAYMENT, BOOKINGS, BOOKING_HISTORY
- **Mô tả:** Trigger thực thi ràng buộc toàn vẹn 2.2.9 — "Booking chỉ được xác nhận khi thanh toán đạt tối thiểu 50% tổng giá trị". Vì trigger cần truy vấn lại bảng payment (chính bảng đang bị thay đổi) để tính tổng tiền đã nộp, nếu dùng trigger thông thường sẽ gặp lỗi mutating table. Do đó, trigger được thiết kế theo dạng **COMPOUND TRIGGER**: giai đoạn AFTER EACH ROW thu thập danh sách booking_id cần kiểm tra; giai đoạn AFTER STATEMENT thực hiện tính toán và xác nhận sau khi toàn bộ câu lệnh DML hoàn tất. Trigger áp dụng công thức để tính tổng giá trị booking:

$$total_price = MAX(raw_price - discount_amount, 0)$$

Trong đó, giá gốc (raw_price) được tính toán đa tầng qua công thức:

$$raw_price = \sum_{i=1}^n (base_price + daily_price_j) \times seasonal_multiplier_k$$

Các bước thực hiện:

- [1] (*AFTER EACH ROW*) Với mỗi dòng trong payment có status = 'SUCCESS', ghi nhận booking_id vào danh sách tạm v_booking_ids.
- [2] (*AFTER STATEMENT*) Sau khi câu lệnh DML hoàn tất, duyệt qua từng booking_id trong danh sách:
 - [2.1] Tính tổng số tiền đã thanh toán thành công (v_total_paid) cho booking đó từ bảng payment.
 - [2.2] Lấy tổng giá trị booking (v_total_price) từ bảng bookings.
 - [2.3] Kiểm tra điều kiện xác nhận:
 - Nếu v_total_paid >= v_total_price / 2 và booking đang ở trạng thái 'PENDING' → cập nhật status = 'CONFIRMED'.
 - Nếu có dòng nào thực sự được cập nhật (SQL%ROWCOUNT > 0) → ghi một bản ghi lịch sử vào booking_history với action = 'CONFIRMED'.
- [3] Xóa danh sách tạm và reset bộ đếm sau mỗi statement.

Mã PL/SQL:

```
CREATE OR REPLACE TRIGGER trg_payment_confirm_booking
```

```
FOR INSERT OR UPDATE ON payment
COMPOUND TRIGGER
```

```
TYPE t_booking_id_list IS TABLE OF payment.booking_id%TYPE
INDEX BY PLS_INTEGER;
```

```
v_booking_ids t_booking_id_list;
v_count      PLS_INTEGER := 0;
```

```
AFTER EACH ROW IS
BEGIN
```

```
IF :NEW.status = 'SUCCESS' THEN
    v_count := v_count + 1;
    v_booking_ids(v_count) := :NEW.booking_id;
END IF;
```

```
END AFTER EACH ROW;
```

```
AFTER STATEMENT IS
```

```
v_total_paid NUMBER;
v_total_price NUMBER;
```

```
BEGIN
```

```
FOR i IN 1 .. v_count LOOP
    SELECT NVL(SUM(amount), 0)
    INTO v_total_paid
    FROM payment
    WHERE booking_id = v_booking_ids(i)
    AND status = 'SUCCESS';
```

```
SELECT total_price
INTO v_total_price
FROM bookings
WHERE id = v_booking_ids(i);
```

```
IF v_total_paid >= v_total_price / 2 THEN
    UPDATE bookings
    SET status = 'CONFIRMED'
    WHERE id = v_booking_ids(i) AND status = 'PENDING';
```

```
IF SQL%ROWCOUNT > 0 THEN
```

```

        INSERT INTO booking_history (booking_id, action, performed_by,
notes)
        VALUES (v_booking_ids(i), 'CONFIRMED', 'SYSTEM', 'Đã xác nhận đặt
phòng');
    END IF;
END IF;
END LOOP;

v_booking_ids.DELETE;
v_count := 0;
END AFTER STATEMENT;

END trg_payment_confirm_booking;

```

d. trg_total_price

- Tên trigger: trg_total_price
- Thao tác: INSERT OR UPDATE
- Bối cảnh: BOOKINGS
- Mô tả: Trigger đảm bảo cột total_price luôn được tính toán chính xác trước khi dữ liệu được ghi vào bảng bookings, theo công thức: $\text{total_price} = \text{GREATEST}(\text{raw_price} - \text{discount_amount}, 0)$. Hàm GREATEST(..., 0) đảm bảo tổng tiền không bao giờ âm trong trường hợp giảm giá vượt quá giá gốc.

Các bước thực hiện:

- [1] Nhận raw_price (giá gốc trước giảm) và discount_amount (số tiền được giảm) từ dòng dữ liệu đang được ghi.
- [2] Tính $\text{total_price} = \text{GREATEST}(:\text{NEW.raw_price} - :\text{NEW.discount_amount}, 0)$.
- [3] Gán giá trị vừa tính vào :NEW.total_price trước khi INSERT/UPDATE hoàn tất.

Mã PL/SQL:

```

CREATE OR REPLACE TRIGGER trg_total_price
BEFORE INSERT OR UPDATE ON bookings
FOR EACH ROW
BEGIN
    :NEW.total_price := GREATEST(:NEW.raw_price - :NEW.discount_amount, 0);
END;

```

3.2. Stored Procedure

3.2.1. Danh sách các Stored Procedure

Bảng 3.2: Danh sách các Stored Procedure được sử dụng trong hệ thống quản lý khách sạn

STT	Tên Stored Procedure	Tham số đầu vào	Tham số đầu ra	Nội dung
1	sp_create_booking	p_customer_id p_room_type_id p_check_in_date p_check_out_date p_promotion_code p_special_requests	o_booking_id	Tạo giao dịch đặt phòng hoàn chỉnh, tuân thủ theo các quy trình như tự động tìm phòng trống, áp dụng khuyến mãi, insert booking và lưu lịch sử
2	sp_update_booking_status	p_booking_id p_new_status p_performed_by p_notes		Thay đổi trạng thái booking (PENDING, CONFIRMED,

				CANCELLED), ghi lại lịch sử, xử lý hoàn trả promotion khi hủy
3	sp_process_check_in	p_booking_id p_performed_by		Xử lý check-in theo quy trình nghiệp vụ check-in, ghi lại lịch sử check-in
4	sp_process_check_out	p_booking_id p_performed_by		Xử lý check-out theo quy trình nghiệp vụ check-out, ghi lại lịch sử check-out
5	sp_generate_daily_sales_report	p_report_date		Tổng hợp doanh thu hàng ngày

3.2.2. Mô tả chi tiết các Stored Procedure

a. sp_create_booking

- Tên Stored Procedure: sp_create_booking
- Tham số đầu vào: p_customer_id, p_room_type_id, p_check_in_date, p_check_out_date, p_promotion_code, p_special_requests
- Tham số đầu ra: o_booking_id
- Mô tả: Procedure đóng gói toàn bộ luồng nghiệp vụ tạo booking: kiểm tra đầu vào, tìm phòng trống, tính giá theo hệ thống 3 tầng, áp dụng mã khuyến mãi nếu có, tạo booking, và ghi lịch sử. Sử dụng SELECT ... FOR UPDATE SKIP

LOCKED để khóa phòng được chọn, tránh hiện tượng Phantom Read (double booking) trong môi trường đồng thời.

Các bước thực hiện:

- [1] Tính số đêm lưu trú ($v_num_nights = p_check_out_date - p_check_in_date$). Nếu số đêm $\leq 0 \rightarrow$ báo lỗi -20100 và hủy giao tác.
- [2] Tìm phòng trống đầu tiên thuộc loại phòng yêu cầu: lọc các phòng không ở trạng thái 'MAINTENANCE' và không có booking xung đột về thời gian (trạng thái PENDING, CONFIRMED, CHECKED_IN). Khóa dòng bằng FOR UPDATE SKIP LOCKED.
 - [2.1] Nếu không tìm được phòng trống $\rightarrow$ báo lỗi -20101 và hủy giao tác.
- [3] Tính giá phòng (raw_price) bằng cách gọi hàm FN_CALCULATE_ROOM_PRICE.
- [4] Nếu có mã khuyến mãi, tính số tiền giảm giá bằng cách gọi FN_CALCULATE_PROMOTION_DISCOUNT.
 - [4.1] Nếu mã không hợp lệ hoặc giảm giá bằng 0 $\rightarrow$ báo lỗi -20102 và hủy giao tác.
- [5] Thực hiện INSERT vào bảng bookings với status = 'PENDING'. Trigger trg_total_price sẽ tự động tính total_price.
- [6] Nếu có mã khuyến mãi, INSERT vào bảng booking_promotions. Trigger trg_bpromo_biu sẽ tự động tăng used_count.
- [7] Ghi lịch sử tạo booking vào bảng booking_history.
- [8] COMMIT giao tác. Nếu có lỗi ở bất kỳ bước nào $\rightarrow$ ROLLBACK và ném lại ngoại lệ.

Mã PL/SQL:

```
CREATE OR REPLACE PROCEDURE sp_create_booking (
  p_customer_id    IN NUMBER,
  p_room_type_id   IN NUMBER,
  p_check_in_date  IN DATE,
  p_check_out_date IN DATE,
```

```

p_promotion_code IN VARCHAR2 DEFAULT NULL,
p_special_requests IN VARCHAR2 DEFAULT NULL,
o_booking_id      OUT NUMBER
)
IS
v_room_id      NUMBER;
v_raw_price     NUMBER(12,2);
v_discount_amount NUMBER(12,2) := 0;
v_num_nights    NUMBER;
v_promotion_id  NUMBER;
BEGIN
v_num_nights := p_check_out_date - p_check_in_date;

IF v_num_nights <= 0 THEN
    RAISE_APPLICATION_ERROR(-20100, 'Ngày check-out phải sau ngày check-
in');
END IF;

BEGIN
    SELECT r.id INTO v_room_id
    FROM rooms r
    WHERE r.room_type_id = p_room_type_id
    AND r.status != 'MAINTENANCE'
    AND NOT EXISTS (
        SELECT 1 FROM bookings b
        WHERE b.room_id = r.id
        AND b.status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
        AND p_check_in_date < b.check_out_date
        AND p_check_out_date > b.check_in_date
    )
    ORDER BY r.floor, r.room_number
    FETCH FIRST 1 ROW ONLY
    FOR UPDATE SKIP LOCKED;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20101,
            'Không còn phòng trống cho loại phòng này trong khoảng thời gian
yêu cầu');
END;

```

```

v_raw_price := FN_CALCULATE_ROOM_PRICE(p_room_type_id,
p_check_in_date, p_check_out_date);

IF p_promotion_code IS NOT NULL THEN
    v_discount_amount := FN_CALCULATE_PROMOTION_DISCOUNT(
        v_raw_price, p_promotion_code, v_num_nights);
    IF v_discount_amount <= 0 THEN
        RAISE_APPLICATION_ERROR(-20102,
            'Mã khuyến mãi không hợp lệ hoặc đã hết hạn: ' || p_promotion_code);
    END IF;
END IF;

INSERT INTO bookings (
    customer_id, room_id, check_in_date, check_out_date,
    raw_price, discount_amount, total_price, status, special_requests
) VALUES (
    p_customer_id, v_room_id, p_check_in_date, p_check_out_date,
    v_raw_price, v_discount_amount, 0, 'PENDING', p_special_requests
) RETURNING id INTO o_booking_id;

IF p_promotion_code IS NOT NULL THEN
    SELECT id INTO v_promotion_id FROM promotions WHERE code =
p_promotion_code;
    INSERT INTO booking_promotions (booking_id, promotion_id)
    VALUES (o_booking_id, v_promotion_id);
END IF;

INSERT INTO booking_history (booking_id, action, performed_by, notes)
VALUES (o_booking_id, 'CREATED', 'SYSTEM',
    'Đặt phòng mới - Loại phòng ID: ' || p_room_type_id ||
    ', Phòng số: ' || v_room_id ||
    ', Từ: ' || TO_CHAR(p_check_in_date, 'DD/MM/YYYY') ||
    ' đến: ' || TO_CHAR(p_check_out_date, 'DD/MM/YYYY'));

COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;

```

```
RAISE;
END SP_CREATE_BOOKING;
```

b. `sp_update_booking_status`

- Tên procedure: `sp_update_booking_status`
- Tham số đầu vào: `p_booking_id`, `p_new_status`, `p_performed_by`, `p_notes`
- Tham số đầu ra: Không có
- Mô tả: Procedure tập trung kiểm soát các chuyển đổi trạng thái hợp lệ của booking, tránh các trường hợp như thay đổi booking đã hủy hoặc đã trả phòng. Sử dụng `SELECT ... FOR UPDATE` để khóa dòng booking trước khi cập nhật, tránh race condition. Khi hủy booking, procedure tự động xóa liên kết với mã khuyến mãi để trigger hoàn trả `used_count`.

Các bước thực hiện:

- [1] Đọc status và `room_id` hiện tại của booking, khóa dòng bằng `FOR UPDATE`.
- [2] Kiểm tra tính hợp lệ của chuyển đổi trạng thái:
 - [2.1] Nếu trạng thái mới trùng trạng thái hiện tại → báo lỗi -20200.
 - [2.2] Nếu booking đang ở trạng thái 'CANCELLED' → báo lỗi -20201.
 - [2.3] Nếu booking đang ở trạng thái 'CHECKED_OUT' → báo lỗi -20202.
- [3] Xác định action sẽ ghi vào `booking_history` dựa trên `p_new_status`.
- [4] Thực hiện `UPDATE` trạng thái booking. Trigger `trg_booking_room_status` tự động cập nhật `rooms.status`.
- [5] Nếu `p_new_status` = 'CANCELLED' → xóa tất cả liên kết trong `booking_promotions` để trigger `trg_bpromo_biu` hoàn trả `used_count`.
- [6] Ghi lịch sử thay đổi vào `booking_history`.
- [7] `COMMIT`. Nếu có lỗi → `ROLLBACK`.

Mã PL/SQL:

```

CREATE OR REPLACE PROCEDURE sp_update_booking_status (
    p_booking_id  IN NUMBER,
    p_new_status  IN VARCHAR2,
    p_performed_by IN VARCHAR2,
    p_notes       IN VARCHAR2 DEFAULT NULL
)
IS
    v_current_status VARCHAR2(20);
    v_room_id        NUMBER;
    v_action         VARCHAR2(50);
BEGIN
    SELECT status, room_id INTO v_current_status, v_room_id
    FROM bookings WHERE id = p_booking_id FOR UPDATE;

    IF v_current_status = p_new_status THEN
        RAISE_APPLICATION_ERROR(-20200, 'Booking đã ở trạng thái: ' ||
p_new_status);
    END IF;
    IF v_current_status = 'CANCELLED' THEN
        RAISE_APPLICATION_ERROR(-20201, 'Không thể thay đổi booking đã bị hủy');
    END IF;
    IF v_current_status = 'CHECKED_OUT' THEN
        RAISE_APPLICATION_ERROR(-20202, 'Không thể thay đổi booking đã trả
phòng');
    END IF;

    CASE p_new_status
        WHEN 'CONFIRMED' THEN v_action := 'CONFIRMED';
        WHEN 'CANCELLED' THEN v_action := 'CANCELLED';
        ELSE                 v_action := 'UPDATED';
    END CASE;

    UPDATE bookings SET status = p_new_status WHERE id = p_booking_id;

    IF p_new_status = 'CANCELLED' THEN
        DELETE FROM booking_promotions WHERE booking_id = p_booking_id;
    END IF;

    INSERT INTO booking_history (booking_id, action, performed_by, notes)

```

```

VALUES (p_booking_id, v_action, p_performed_by,
        NVL(p_notes, 'Cập nhật trạng thái: ' || v_current_status || ' -> ' || p_new_status));

COMMIT;
EXCEPTION
    WHEN OTHERS THEN ROLLBACK; RAISE;
END SP_UPDATE_BOOKING_STATUS;

```

c. sp_process_check_in

- Tên procedure: sp_process_check_in
- Tham số đầu vào: p_booking_id, p_performed_by
- Tham số đầu ra: Không có
- Mô tả: Procedure xử lý nghiệp vụ check-in khách hàng, bao gồm kiểm tra điều kiện hợp lệ và cập nhật trạng thái. Trigger trg_booking_room_status tự động chuyển phòng sang trạng thái 'OCCUPIED' sau khi booking được cập nhật thành 'CHECKED_IN'.

Các bước thực hiện:

- [1] Đọc status, check_in_date, room_id của booking, khóa dòng bằng FOR UPDATE.
- [2] Kiểm tra trạng thái booking:
 - [2.1] Nếu status != 'CONFIRMED' → báo lỗi -20300 và hủy giao tác.
- [3] Kiểm tra ngày check-in (cho phép check-in sớm tối đa 1 ngày):
 - [3.1] Nếu ngày hiện tại < check_in_date - 1 → báo lỗi -20301 và hủy giao tác.
- [4] Cập nhật bookings.status = 'CHECKED_IN'. Trigger trg_booking_room_status tự động cập nhật rooms.status = 'OCCUPIED'.
- [5] Ghi lịch sử check-in vào booking_history.
- [6] COMMIT. Nếu có lỗi → ROLLBACK.

Mã PL/SQL:

```

CREATE OR REPLACE PROCEDURE sp_process_check_in (
    p_booking_id IN NUMBER,
    p_performed_by IN VARCHAR2
)
IS
    v_current_status VARCHAR2(20);
    v_check_in_date DATE;
    v_room_id NUMBER;
BEGIN

    SELECT status, check_in_date, room_id
    INTO v_current_status, v_check_in_date, v_room_id
    FROM bookings
    WHERE id = p_booking_id
    FOR UPDATE;

    IF v_current_status != 'CONFIRMED' THEN
        RAISE_APPLICATION_ERROR(-20300,
            'Chỉ có thể check-in khi booking ở trạng thái CONFIRMED. Trạng thái
            hiện tại: ' || v_current_status);
    END IF;

    IF TRUNC(SYSDATE) < TRUNC(v_check_in_date) - 1 THEN
        RAISE_APPLICATION_ERROR(-20301,
            'Chưa đến ngày check-in. Ngày check-in: ' || TO_CHAR(v_check_in_date,
            'DD/MM/YYYY'));
    END IF;

    UPDATE bookings
    SET status = 'CHECKED_IN'
    WHERE id = p_booking_id;

    INSERT INTO booking_history (
        booking_id, action, performed_by, notes
    ) VALUES (
        p_booking_id, 'UPDATED', p_performed_by,
        'Check-in thành công - Phòng ID: ' || v_room_id ||

```

```

    ' - Thời gian: ' || TO_CHAR(SYSTIMESTAMP, 'DD/MM/YYYY HH24:MI:SS')
);

COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END SP_PROCESS_CHECK_IN;
\

```

d. `sp_process_check_out`

- Tên procedure: `sp_process_check_out`
- Tham số đầu vào: `p_booking_id`, `p_performed_by`
- Tham số đầu ra: Không có
- Mô tả: Procedure xử lý nghiệp vụ trả phòng, đặc biệt bao gồm bước kiểm tra thanh toán đầy đủ trước khi cho phép check-out. Sau khi cập nhật trạng thái, trigger `trg_booking_room_status` tự động chuyển phòng sang 'MAINTENANCE' để chuẩn bị dọn dẹp.

Các bước thực hiện:

- [1] Đọc `status`, `room_id`, `total_price` của booking, khóa dòng bằng FOR UPDATE.
- [2] Kiểm tra trạng thái booking:
 - [2.1] Nếu `status != 'CHECKED_IN'` → báo lỗi -20400 và hủy giao tác.
- [3] Tính tổng số tiền đã thanh toán thành công (`v_total_paid`) từ bảng `payment`.
- [4] Kiểm tra thanh toán:
 - [4.1] Nếu `v_total_paid < total_price` → báo lỗi -20401 kèm chi tiết số tiền còn thiếu và hủy giao tác.
- [5] Cập nhật `bookings.status = 'CHECKED_OUT'`. Trigger `trg_booking_room_status` tự động cập nhật `rooms.status = 'MAINTENANCE'`.

- [6] Ghi lịch sử check-out kèm tổng thanh toán vào booking_history.
- [7] COMMIT. Nếu có lỗi → ROLLBACK.

Mã PL/SQL:

```

CREATE OR REPLACE PROCEDURE sp_process_check_out (
    p_booking_id  IN NUMBER,
    p_performed_by IN VARCHAR2
)
IS
    v_current_status VARCHAR2(20);
    v_room_id        NUMBER;
    v_total_price     NUMBER(12,2);
    v_total_paid      NUMBER(12,2);
BEGIN
    SELECT status, room_id, total_price
    INTO   v_current_status, v_room_id, v_total_price
    FROM   bookings
    WHERE  id = p_booking_id
    FOR UPDATE;

    IF v_current_status != 'CHECKED_IN' THEN
        RAISE_APPLICATION_ERROR(-20400,
            'Chỉ có thể check-out khi booking ở trạng thái CHECKED_IN. Trạng thái
hiện tại: ' || v_current_status);
    END IF;

    SELECT NVL(SUM(amount), 0)
    INTO   v_total_paid
    FROM   payment
    WHERE  booking_id = p_booking_id
    AND    status = 'SUCCESS';

    IF v_total_paid < v_total_price THEN
        RAISE_APPLICATION_ERROR(-20401,
            'Khách chưa thanh toán đủ. Tổng tiền: ' || v_total_price ||
            ', Đã thanh toán: ' || v_total_paid ||
            ', Còn thiếu: ' || (v_total_price - v_total_paid));
    
```

```

END IF;

UPDATE bookings
SET   status = 'CHECKED_OUT'
WHERE id = p_booking_id;

INSERT INTO booking_history (
    booking_id, action, performed_by, notes
) VALUES (
    p_booking_id, 'UPDATED', p_performed_by,
    'Check-out thành công - Phòng ID: ' || v_room_id ||
    ' - Tổng thanh toán: ' || v_total_paid ||
    ' - Thời gian: ' || TO_CHAR(SYSTIMESTAMP, 'DD/MM/YYYY HH24:MI:SS')
);

COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END SP_PROCESS_CHECK_OUT;
/

```

e. `sp_generate_daily_report`

- Tên procedure: `sp_generate_daily_report`
- Tham số đầu ra: `p_report_date`
- Mô tả: Procedure tổng hợp dữ liệu doanh thu theo ngày và lưu vào bảng `sales_reports`. Có thể được lên lịch tự động bằng Oracle DBMS_SCHEDULER chạy mỗi ngày lúc 23:59. Nếu báo cáo cho ngày đó đã tồn tại (chạy lại), procedure sẽ cập nhật thay vì tạo mới (upsert logic).

Các bước thực hiện:

- [1] Tổng hợp từ bảng `bookings` theo `check_out_date = p_report_date` và status IN ('CHECKED_OUT', 'CHECKED_IN'): đếm tổng số booking, tổng doanh

thu gộc (gross_revenue), tổng giảm giá (discount_amount), doanh thu ròng (net_revenue).

- [2] Tính tỷ lệ lấp đầy phòng (occupancy_rate):
 - [2.1] Đếm tổng số phòng đang hoạt động (không ở trạng thái 'MAINTENANCE').
 - [2.2] Đếm số phòng có khách trong ngày báo cáo (booking CONFIRMED hoặc CHECKED_IN bao phủ ngày đó).
 - [2.3] Tính occupancy_rate = ROUND(v_occupied_rooms / v_total_rooms * 100, 2).
- [3] Kiểm tra xem đã có báo cáo cho ngày này chưa:
 - [3.1] Nếu đã có → UPDATE báo cáo hiện có.
 - [3.2] Nếu chưa có → INSERT báo cáo mới.
- [4] COMMIT. Nếu có lỗi → ROLLBACK.

Mã PL/SQL:

```
CREATE OR REPLACE PROCEDURE sp_generate_daily_sales_report (
  p_report_date IN DATE
)
IS
  v_total_bookings    NUMBER(10,0) := 0;
  v_gross_revenue     NUMBER(15,2) := 0;
  v_discount_amount   NUMBER(15,2) := 0;
  v_net_revenue       NUMBER(15,2) := 0;
  v_total_rooms       NUMBER;
  v_occupied_rooms    NUMBER;
  v_occupancy_rate    NUMBER(5,2) := 0;
  v_report_exists     NUMBER;
BEGIN
  SELECT NVL(COUNT(*), 0),
         NVL(SUM(raw_price), 0),
         NVL(SUM(discount_amount), 0),
         NVL(SUM(total_price), 0)
  INTO v_total_bookings,
```

```

        v_gross_revenue,
        v_discount_amount,
        v_net_revenue
FROM   bookings
WHERE  TRUNC(check_out_date) = TRUNC(p_report_date)
      AND status IN ('CHECKED_OUT', 'CHECKED_IN');

SELECT COUNT(*)
INTO   v_total_rooms
FROM   rooms
WHERE  status != 'MAINTENANCE';

SELECT COUNT(DISTINCT room_id)
INTO   v_occupied_rooms
FROM   bookings
WHERE  status IN ('CONFIRMED', 'CHECKED_IN')
      AND TRUNC(p_report_date) BETWEEN TRUNC(check_in_date) AND
TRUNC(check_out_date) - 1;

IF v_total_rooms > 0 THEN
    v_occupancy_rate := ROUND((v_occupied_rooms / v_total_rooms) * 100, 2);
END IF;

SELECT COUNT(*)
INTO   v_report_exists
FROM   sales_reports
WHERE  report_date = TRUNC(p_report_date);

IF v_report_exists > 0 THEN

    UPDATE sales_reports
    SET   total_bookings    = v_total_bookings,
          gross_revenue     = v_gross_revenue,
          discount_amount   = v_discount_amount,
          net_revenue       = v_net_revenue,
          room_occupancy_rate = v_occupancy_rate,

```

```

        created_at      = CURRENT_TIMESTAMP
    WHERE report_date = TRUNC(p_report_date);
ELSE

    INSERT INTO sales_reports (
        report_date, total_bookings, gross_revenue,
        discount_amount, net_revenue, room_occupancy_rate
    ) VALUES (
        TRUNC(p_report_date), v_total_bookings, v_gross_revenue,
        v_discount_amount, v_net_revenue, v_occupancy_rate
    );
END IF;

COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END SP_GENERATE_DAILY_SALES_REPORT;
/

```

3.3 Stored Function

3.3.1 Danh sách các Stored Function

STT	Tên Stored Function	Tham số đầu vào	Kiểu dữ liệu đầu ra	Nội dung
1	fn_calculate_room_price	p_room_type_id p_check_in_date p_check_out_date	NUMBER (raw_price)	Tính tổng tiền phòng cho một khoảng thời gian đặt phòng theo hệ ưu tiên 3 tầng: daily_prices

				> seasonal_prices > base_price
2	fn_check_room_availability	p_room_id p_check_in_date p_check_out_date p_exclude_booking_id	NUMBER (1 = trống, 0 = bận)	Kiểm tra một phòng cụ thể có trống trong khoảng thời gian yêu cầu không; hỗ trợ loại trừ booking hiện tại khi cập nhật
3	fn_get_available_room_count	p_room_type_id p_check_in_date p_check_out_date	NUMBER (số phòng trống)	Đếm số lượng phòng còn trống của một loại phòng trong khoảng thời gian; dùng để hiển thị trên giao diện tìm kiếm phòng
4	fn_calculate_promotion_discount	p_raw_price p_promotion_code p_num_nights	NUMBER (số tiền giảm)	Tính số tiền giảm giá từ mã khuyến mãi; kiểm tra hiệu lực, giới hạn lượt

				dùng và số đêm tối thiểu trước khi tính
--	--	--	--	--

3.3.2 Mô tả chi tiết các Stored Function

a. fn_calculate_room_price

- Tên function: fn_calculate_room_price
- Tham số đầu vào: p_room_type_id, p_check_in_date, p_check_out_date
- Kiểu dữ liệu đầu ra: NUMBER
- Mô tả: Function tính tổng giá phòng bằng cách cộng giá từng ngày lưu trú (từ ngày check-in đến ngày check-out trừ 1). Mỗi ngày được định giá theo hệ thống ưu tiên 3 tầng nhằm đảm bảo tính linh hoạt trong quản lý giá khách sạn.

Hệ thống ưu tiên giá:

- Ưu tiên 1 — Giá đặc biệt theo ngày (daily_prices): Giá được thiết lập riêng cho từng ngày cụ thể (ngày lễ, sự kiện đặc biệt). Giá này có độ ưu tiên cao nhất.
- Ưu tiên 2 — Hệ số giá theo mùa (seasonal_prices): Nếu ngày hiện tại nằm trong một khoảng mùa định nghĩa, giá được tính bằng $\text{base_price} \times \text{price_multiplier}$. Nếu trùng nhiều mùa, lấy mùa có priority cao nhất.
- Ưu tiên 3 — Giá cơ bản (room_types.base_price): Áp dụng khi không có giá theo ngày hay mùa.

Các bước thực hiện:

- [1] Lấy base_price của loại phòng từ bảng room_types.
- [2] Khởi tạo biến $v_current_date = p_check_in_date$, $v_total_price = 0$.
- [3] Lặp qua từng ngày trong khoảng $[check_in_date, check_out_date)$:
 - [3.1] Tìm giá trong bảng daily_prices cho ngày hiện tại.
 - Nếu có $\rightarrow$ gán v_day_price bằng giá đặc biệt. Chuyển sang bước [3.4].
 - [3.2] Nếu không có giá ngày đặc biệt $\rightarrow$ tìm hệ số giá mùa trong seasonal_prices (lấy theo priority DESC).

- Nếu có → gán $v_day_price = base_price \times price_multiplier$.
Chuyển sang bước [3.4].
- [3.3] Nếu không có giá mùa → gán $v_day_price = base_price$.
- [3.4] Cộng v_day_price vào v_total_price .
- [3.5] Tăng $v_current_date$ lên 1 ngày. Quay lại bước [3].
- [4] Trả về v_total_price .

Mã PL/SQL:

```

CREATE OR REPLACE FUNCTION fn_calculate_room_price (
  p_room_type_id IN NUMBER,
  p_check_in_date IN DATE,
  p_check_out_date IN DATE
) RETURN NUMBER
IS
  v_total_price NUMBER(12,2) := 0;
  v_day_price   NUMBER(12,2);
  v_base_price  NUMBER(12,2);
  v_current_date DATE;
  v_daily_price NUMBER(12,2);
  v_multiplier  NUMBER(10,2);
BEGIN
  SELECT base_price INTO v_base_price
  FROM room_types WHERE id = p_room_type_id;

  v_current_date := p_check_in_date;

  WHILE v_current_date < p_check_out_date LOOP
    v_day_price := NULL;

    -- Ưu tiên 1: Giá đặc biệt theo ngày
    BEGIN
      SELECT price INTO v_daily_price
      FROM daily_prices
      WHERE room_type_id = p_room_type_id AND price_date = v_current_date;
      v_day_price := v_daily_price;
    EXCEPTION WHEN NO_DATA_FOUND THEN NULL;

```

```

END;

-- Ưu tiên 2: Hệ số giá theo mùa
IF v_day_price IS NULL THEN
  BEGIN
    SELECT price_multiplier INTO v_multiplier
    FROM seasonal_prices
    WHERE room_type_id = p_room_type_id
      AND v_current_date BETWEEN start_date AND end_date
    ORDER BY priority DESC FETCH FIRST 1 ROW ONLY;
    v_day_price := v_base_price * v_multiplier;
  EXCEPTION WHEN NO_DATA_FOUND THEN NULL;
  END;
END IF;

-- Ưu tiên 3: Giá cơ bản
IF v_day_price IS NULL THEN
  v_day_price := v_base_price;
END IF;

v_total_price := v_total_price + v_day_price;
v_current_date := v_current_date + 1;
END LOOP;

RETURN v_total_price;
END FN_CALCULATE_ROOM_PRICE;

```

b. fn_check_room_availability

- Tên function: fn_check_room_availability
- Tham số đầu vào: p_room_id, p_check_in_date, p_check_out_date, p_exclude_booking_id
- Kiểu dữ liệu đầu ra: NUMBER (trả về 0 nếu phòng đã được đặt, trả về 1 nếu phòng trống)
- Mô tả: Function kiểm tra một phòng cụ thể có khả dụng để đặt trong khoảng thời gian yêu cầu không. Điều kiện xung đột thời gian giữa hai khoảng [A, B) và [C,

D) là $A < D \text{ AND } C < B$. Tham số `p_exclude_booking_id` được dùng khi cập nhật một booking hiện có — để tránh booking đó tự xung đột với chính nó.

Các bước thực hiện:

- [1] Lấy trạng thái (status) của phòng từ bảng `rooms`.
 - [1.1] Nếu phòng đang ở trạng thái 'MAINTENANCE' → trả về 0 (không khả dụng).
- [2] Đếm số booking đang hoạt động (PENDING, CONFIRMED, CHECKED_IN) của phòng này có thời gian xung đột với khoảng yêu cầu (loại trừ `p_exclude_booking_id` nếu có).
- [3] Kiểm tra kết quả:
 - [3.1] Nếu có xung đột ($v_conflict_count > 0$) → trả về 0.
 - [3.2] Nếu không có xung đột → trả về 1.

Mã PL/SQL:

```
CREATE OR REPLACE FUNCTION fn_check_room_availability (
  p_room_id      IN NUMBER,
  p_check_in_date IN DATE,
  p_check_out_date IN DATE,
  p_exclude_booking_id IN NUMBER DEFAULT NULL
) RETURN NUMBER
IS
  v_conflict_count NUMBER;
  v_room_status   VARCHAR2(20);
BEGIN
  SELECT status INTO v_room_status FROM rooms WHERE id = p_room_id;

  IF v_room_status = 'MAINTENANCE' THEN
    RETURN 0;
  END IF;

  SELECT COUNT(*) INTO v_conflict_count
  FROM bookings
  WHERE room_id = p_room_id
  AND status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN');
```

```

AND (p_exclude_booking_id IS NULL OR id != p_exclude_booking_id)
AND p_check_in_date < check_out_date
AND p_check_out_date > check_in_date;

IF v_conflict_count > 0 THEN RETURN 0;
ELSE RETURN 1;
END IF;
END FN_CHECK_ROOM_AVAILABILITY;
\

```

c. fn_get_available_room_count

- Tên function: fn_get_available_room_count
- Tham số đầu vào: p_room_type_id, p_check_in_date, p_check_out_date
- Kiểu dữ liệu đầu ra: NUMBER
- Mô tả: Function được sử dụng ở tầng hiển thị giao diện khách hàng để thông báo số phòng còn trống khi tìm kiếm. Thay vì kiểm tra từng phòng riêng lẻ, function dùng một câu truy vấn với NOT EXISTS để đếm trực tiếp tất cả phòng đủ điều kiện.

Các bước thực hiện:

- [1] Đếm tất cả phòng thuộc loại p_room_type_id thỏa mãn đồng thời:
 - [1.1] Không ở trạng thái 'MAINTENANCE'.
 - [1.2] Không có booking nào (PENDING, CONFIRMED, CHECKED_IN) xung đột về thời gian với khoảng yêu cầu.
- [2] Trả về số đếm tìm được.

Mã PL/SQL:

```

CREATE OR REPLACE FUNCTION fn_get_available_room_count (
  p_room_type_id IN NUMBER,
  p_check_in_date IN DATE,
  p_check_out_date IN DATE
) RETURN NUMBER
IS
  v_available_count NUMBER := 0;

```

```

BEGIN
    SELECT COUNT(*)
    INTO v_available_count
    FROM rooms r
    WHERE r.room_type_id = p_room_type_id
    AND r.status != 'MAINTENANCE'
    AND NOT EXISTS (
        SELECT 1
        FROM bookings b
        WHERE b.room_id = r.id
        AND b.status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
        AND p_check_in_date < b.check_out_date
        AND p_check_out_date > b.check_in_date
    );

    RETURN v_available_count;
END FN_GET_AVAILABLE_ROOM_COUNT;
/

```

d. fn_calculate_promotion_discount

- Tên function: fn_calculate_promotion_discount
- Tham số đầu vào: p_raw_price, p_promotion_code, p_num_nights
- Kiểu dữ liệu đầu ra: NUMBER
- Mô tả: Function kiểm tra toàn diện tính hợp lệ của mã khuyến mãi trước khi tính số tiền giảm. Hệ thống hỗ trợ hai loại giảm giá: theo phần trăm (PERCENTAGE) và số tiền cố định (FIXED). Function trả về 0 thay vì ném ngoại lệ khi mã không hợp lệ, giúp tăng gọi dễ xử lý.

Các bước thực hiện:

- [1] Nếu p_promotion_code là NULL → trả về 0.
- [2] Tìm thông tin mã khuyến mãi trong bảng promotions.
 - [2.1] Nếu không tìm thấy → trả về 0.
- [3] Kiểm tra thời hạn hiệu lực:

- [3.1] Nếu ngày hiện tại nằm ngoài khoảng [start_date, end_date] → trả về 0.
- [4] Kiểm tra giới hạn lượt sử dụng:
 - [4.1] Nếu max_uses không NULL và used_count >= max_uses → trả về 0.
- [5] Kiểm tra điều kiện số đêm tối thiểu:
 - [5.1] Nếu min_nights không NULL và p_num_nights < min_nights → trả về 0.
- [6] Tính số tiền giảm giá:
 - [6.1] Nếu loại 'PERCENTAGE' → discount = raw_price × (discount_value / 100).
 - [6.2] Nếu loại 'FIXED' → discount = LEAST(discount_value, raw_price) (không giảm quá giá gốc).
- [7] Trả về ROUND(discount_amount, 2).

Mã PL/SQL:

```
CREATE OR REPLACE FUNCTION fn_calculate_promotion_discount (
    p_raw_price    IN NUMBER,
    p_promotion_code IN VARCHAR2,
    p_num_nights   IN NUMBER DEFAULT NULL
) RETURN NUMBER
IS
    v_discount_amount NUMBER(12,2) := 0;
    v_discount_type   VARCHAR2(20);
    v_discount_value  NUMBER(12,2);
    v_start_date      DATE;
    v_end_date        DATE;
    v_min_nights      NUMBER(10,0);
    v_max_uses        NUMBER(10,0);
    v_used_count      NUMBER(10,0);
BEGIN
    IF p_promotion_code IS NULL THEN RETURN 0; END IF;
```

```

BEGIN
    SELECT discount_type, discount_value, start_date, end_date,
           min_nights, max_uses, used_count
    INTO v_discount_type, v_discount_value, v_start_date, v_end_date,
         v_min_nights, v_max_uses, v_used_count
    FROM promotions WHERE code = p_promotion_code;
EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 0;
END;

IF TRUNC(SYSDATE) < v_start_date OR TRUNC(SYSDATE) > v_end_date THEN
    RETURN 0;
END IF;

IF v_max_uses IS NOT NULL AND v_used_count >= v_max_uses THEN
    RETURN 0;
END IF;

IF v_min_nights IS NOT NULL AND p_num_nights IS NOT NULL THEN
    IF p_num_nights < v_min_nights THEN RETURN 0; END IF;
END IF;

IF v_discount_type = 'PERCENTAGE' THEN
    v_discount_amount := p_raw_price * (v_discount_value / 100);
ELSIF v_discount_type = 'FIXED' THEN
    v_discount_amount := LEAST(v_discount_value, p_raw_price);
END IF;

RETURN ROUND(v_discount_amount, 2);
END FN_CALCULATE_PROMOTION_DISCOUNT;

```

Chương 4: XỬ LÝ TRUY XUẤT ĐỒNG THỜI

4.1 Các mức cô lập trong Oracle

Trong hệ thống quản lý khách sạn, nhiều người dùng (khách hàng, quản lý, lễ tân) có thể truy cập và thao tác trên cùng một dữ liệu tại cùng một thời điểm. Oracle đảm bảo tính toàn vẹn thông qua cơ chế MVCC (Multi-Version Concurrency Control) – mỗi giao tác nhìn thấy một Snapshot nhất quán của dữ liệu, thay vì khóa toàn bộ bảng để chờ nhau.

Có 4 hiện tượng không mong muốn có thể xảy ra khi truy xuất đồng thời:

- Dirty Read: Đọc dữ liệu chưa được commit của giao tác khác
- Non-Repeatable Read: Đọc cùng một dòng hai lần, kết quả khác nhau do giao tác khác UPDATE
- Phantom Read: Đọc cùng một tập dữ liệu hai lần, kết quả khác nhau do giao tác khác INSERT/DELETE
- Lost Update: Hai giao tác cùng sửa một dữ liệu, cập nhật của giao tác này bị ghi đè bởi giao tác kia

Hiện tại, Oracle hỗ trợ 2 mức cô lập chính: Read Committed, Serializable và mức cô lập không được hỗ trợ chính thức như một mức cô lập riêng biệt là Repeatable Read.

4.1.1 Read Committed (mặc định)

Read Committed được định nghĩa là một giao tác chỉ đọc được dữ liệu đã được commit bởi giao tác khác. Mỗi câu lệnh SELECT nhận một Snapshot mới nhất tại thời điểm câu lệnh đó bắt đầu thực thi (không phải thời điểm giao tác bắt đầu).

Ưu điểm:

- Hiệu năng cao, phù hợp với đa số hệ thống
- Không bị Dirty Read
- Dễ tránh Deadlock hơn các mức cô lập cao hơn

Nhược điểm:

- Có thể xảy ra Non-Repeatable Read và Phantom Read

4.1.2 *Serializable*

Serializable được định nghĩa là giao tác nhận một Snapshot cố định tại thời điểm giao tác bắt đầu. Mọi câu lệnh trong cùng giao tác đều đọc dữ liệu nhất quán từ snapshot đó, bất kể các giao tác khác có commit thay đổi hay không trong thời gian đó.

Ưu điểm:

- Loại bỏ được Non-Repeatable Read và Phantom Read, đảm bảo tính nhất quán cao.
- Dễ suy luận logic chương trình hơn
- Tránh các bất thường khi thao tác tập dữ liệu

Nhược điểm:

- Có thể phát sinh lỗi ORA-08177: can't serialize access khi giao tác khác sửa đổi dữ liệu mà giao tác hiện tại đã đọc
- Hiệu năng thấp hơn Read Committed

4.1.3 *Repeatable Read*

Mức cô lập Repeatable Read đảm bảo rằng nếu một giao tác đọc một dòng dữ liệu, các lần đọc tiếp theo của cùng dòng đó trong cùng giao tác sẽ luôn trả về kết quả giống nhau – ngay cả khi giao tác khác đã UPDATE và COMMIT dữ liệu đó. Tuy nhiên, Phantom Read vẫn có thể xảy ra (các dòng mới được INSERT/DELETE bởi giao tác khác vẫn có thể xuất hiện hoặc biến mất).

Tính đến thời điểm hiện tại, Oracle không hỗ trợ Repeatable Read như một mức cô lập riêng biệt theo cú pháp SET TRANSACTION ISOLATION LEVEL REPEATABLE READ. Tuy nhiên, Oracle đạt được hiệu ứng tương đương thông qua hai cơ chế:

- **Cơ chế 1 - MVCC tự nhiên (Statement-level consistency):**
Trong mức Read Committed, mỗi câu lệnh đã nhất quán với một snapshot. Để đọc lại cùng dữ liệu nhất quán trong nhiều câu lệnh, Oracle dùng mức Serializable.
- **Cơ chế 2 – SELECT ... FOR UPDATE (Row-level locking):**
Đây là cách Oracle thực tế đạt được Repeatable Read cho các nghiệp vụ quan trọng. Khi một giao tác đọc dòng dữ liệu bằng FOR UPDATE, không giao tác

nào khác có thể UPDATE dòng đó cho đến khi giao tác hiện tại kết thúc – đảm bảo kết quả đọc không thay đổi.

Ưu điểm:

- Ngăn chặn được Non-Repeatable Read
- Giảm thiểu nguy cơ xung đột dữ liệu
- Đảm bảo tính nhất quán khi đọc nhiều lần

Nhược điểm:

- Không ngăn chặn được Phantom Read
- Khóa dữ liệu lâu hơn và hiệu năng thấp hơn Read Committed
- Có thể gây Deadlock nhiều hơn

4.2 Cơ chế khóa (Locking)

Trong Oracle Database, cơ chế khóa đóng vai trò quan trọng trong việc duy trì tính nhất quán và toàn vẹn dữ liệu khi nhiều giao dịch cùng truy cập vào một nguồn dữ liệu chung. Oracle triển khai hệ thống khóa với nhiều loại khác nhau, tiêu biểu như share lock và exclusive lock, nhằm kiểm soát quyền truy cập và hạn chế xung đột giữa các giao dịch.

Oracle sử dụng Row-level Locking (khóa ở cấp độ dòng) thay vì table-level locking, giúp tối đa hóa khả năng truy xuất đồng thời. Khi một giao tác cập nhật một dòng, chỉ dòng đó bị khóa – các dòng khác trong cùng bảng vẫn có thể được truy cập tự do bởi các giao tác khác.

Oracle tự động cấp khóa exclusive khi thực thi DML (INSERT, UPDATE, DELETE) và giải phóng khóa khi giao tác COMMIT hoặc ROLLBACK. Ngoài ra, lệnh SELECT ... FOR UPDATE cho phép giao tác chủ động yêu cầu khóa dòng trước khi thực hiện thao tác, ngăn chặn các giao tác khác thay đổi dữ liệu đó.

4.3 Deadlock

Trong Oracle Database, deadlock là trạng thái xảy ra khi hai hoặc nhiều giao dịch cùng chờ nhau giải phóng tài nguyên đang bị khóa, tạo thành một vòng phụ thuộc khiến không giao dịch nào có thể tiếp tục thực thi.

Khi phát hiện deadlock, Oracle sẽ tự động xử lý bằng cách hủy một trong các giao dịch liên quan nhằm phá vỡ vòng chờ và giải phóng tài nguyên bị khóa. Cơ chế này giúp hệ thống tránh rơi vào tình trạng tắc nghẽn kéo dài, đồng thời duy trì sự ổn định và đảm bảo tính nhất quán của dữ liệu.

Kịch bản Deadlock trong hệ thống quản lý khách sạn:

Trong hệ thống quản lý khách sạn, kịch bản điển hình xảy ra khi hai lễ tân đồng thời xử lý đặt phòng cho khách, nhưng thứ tự khóa phòng lại ngược nhau. Giả sử trước khi áp dụng SKIP LOCKED:

- Giao tác A (Lễ tân 1 đặt phòng 101 trước, rồi 102):
 - T=1 : SELECT ... FOR UPDATE phòng 101 → Khóa thành công
 - T=2 : SELECT ... FOR UPDATE phòng 102 → CHỜ (đang bị B khóa)
- Giao tác B (Lễ tân 2 đặt phòng 102 trước, rồi 101):
 - T=1 : SELECT ... FOR UPDATE phòng 102 → Khóa thành công
 - T=2 : SELECT ... FOR UPDATE phòng 101 → CHỜ (đang bị A khóa)

Như vậy, hệ thống sẽ bị Deadlock. Giải pháp chính là sử dụng SKIP LOCKED trong sp_create_booking. Thay vì chờ đợi phòng đang bị khóa, giao tác tự động bỏ qua và chọn phòng trống khác:

```
SELECT r.id INTO v_room_id
FROM rooms r
WHERE r.room_type_id = p_room_type_id
  AND r.status != 'MAINTENANCE'
  AND NOT EXISTS ( ... )
ORDER BY r.floor, r.room_number
FETCH FIRST 1 ROW ONLY
FOR UPDATE SKIP LOCKED;
```

4.4 Mô tả các hiện tượng đồng thời trong hệ thống

4.4.1 Phantom Read – Hai phiên cùng tìm phòng trống

Phantom Read xảy ra khi một giao tác đọc một tập dữ liệu hai lần và nhận được kết quả khác nhau, do một giao tác khác đã INSERT hoặc DELETE dòng dữ liệu thỏa mãn điều kiện tìm kiếm, giữa hai lần đọc đó.

Kịch bản trong hệ thống quản lý khách sạn:

- T=1: Khách A truy vấn: "Có bao nhiêu phòng Deluxe trống từ 25/5 đến 28/5?"
[Phiên A - Read Committed]
 - Kết quả: 3 phòng trống (101, 102, 103)
 - Giao diện hiển thị "Còn 3 phòng Deluxe"
- T=2: Lễ tân đặt phòng 103 cho khách B, COMMIT thành công
[Phiên Lễ tân - SP_CREATE_BOOKING hoàn tất]
- T=3: Khách A nhấn "Đặt phòng", hệ thống truy vấn lại (cùng loại query):
 - Kết quả: 2 phòng trống (101, 102), phòng 103 không xuất hiện trong truy vấn. Đây chính là trường hợp Phantom Read.

Bảng mô tả Phantom Read:*Bảng ...: Bảng mô tả phantom read*

Session 1 (S1) – Khách A	Session 2 (S2) – Lễ tân	Giải thích
-- Bước 1: Tìm phòng Deluxe trống		
<pre>SELECT fn_get_available_room_count(1, DATE '2025-05-25', DATE '2025-05-28') FROM dual;</pre>		S1 truy vấn số phòng Deluxe còn trống trong khoảng 25–28/5
-- Kết quả: 3		S1 nhận được 3 phòng trống (101, 102, 103)

	-- Bước 2: Lễ tân đặt phòng 103	
	<pre>EXEC sp_create_booking(p_custom er_id => 5, p_room_type_id => 1, p_check_in_date => DATE '2025-05-25', p_check_out_date => DATE '2025-05-28', o_booking_id => :v_id);</pre>	S2 chạy procedure đặt phòng 103 cho khách B
	COMMIT;	S2 commit thành công – booking phòng 103 được xác nhận
-- Bước 3: Khách A truy vấn lại		
<pre>SELECT fn_get_available_room_count(1, DATE '2025-05-25', DATE '2025- 05-28') FROM dual;</pre>		S1 chạy lại cùng câu truy vấn trong cùng giao tác
-- Kết quả: 2 (Phantom Read)		S1 chỉ còn thấy 2 phòng – phòng 103 đã "biến mất" do S2 vừa INSERT

		booking và COMMIT
--	--	----------------------

Giải pháp áp dụng: Hàm fn_get_available_room_count và sp_create_booking đều kiểm tra lại tính sẵn có của phòng bằng điều kiện NOT EXISTS ngay trước khi INSERT booking, kết hợp FOR UPDATE SKIP LOCKED để khóa phòng được chọn – đảm bảo không thể có hai booking cùng phòng dù Phantom Read xảy ra ở tầng hiển thị.

```

SELECT r.id
INTO v_room_id
FROM rooms r
WHERE r.room_type_id = p_room_type_id
AND r.status != 'MAINTENANCE'
AND NOT EXISTS (
    SELECT 1
    FROM bookings b
    WHERE b.room_id = r.id
    AND b.status IN ('PENDING', 'CONFIRMED', 'CHECKED_IN')
    AND p_check_in_date < b.check_out_date
    AND p_check_out_date > b.check_in_date
)
ORDER BY r.floor, r.room_number
FETCH FIRST 1 ROW ONLY
FOR UPDATE SKIP LOCKED;

```

4.4.2 Lost Update – Cập nhật trạng thái 2 phòng đồng thời

Lost Update xảy ra khi hai giao tác cùng đọc một giá trị, cùng sửa đổi dựa trên giá trị đó, và một trong hai lần ghi ghi đè lên lần ghi kia – khiến một cập nhật bị "mất" hoàn toàn.

Kịch bản trong hệ thống:

- T=1: Lễ tân A đọc booking #15: status = 'PENDING'
- T=1: Lễ tân B đọc booking #15: status = 'PENDING'
- (Cả hai đọc cùng một snapshot – không có khóa)
- T=2: Lễ tân A cập nhật booking #15: PENDING → CONFIRMED, COMMIT

- T=2: Lễ tân B cập nhật booking #15: PENDING → CANCELLED, COMMIT (Lễ tân B vẫn nghĩ booking đang PENDING vì đọc giá trị cũ)
- Kết quả: booking #15 = 'CANCELLED'
- Thao tác CONFIRM của Lễ tân A bị mất.

Mô tả Lost Update (khi chưa có FOR UPDATE):

Bảng ???: Mô tả Lost Update khi chưa có FOR UPDATE

Session 1 (S1) – Lễ tân A	Session 2 (S2) – Lễ tân B	Giải thích
-- Bước 1: Đọc trạng thái booking		
SELECT status FROM bookings WHERE id = 15;		S1 đọc trạng thái booking #15
-- Kết quả: PENDING		S1 thấy booking đang ở trạng thái PENDING
	SELECT status FROM bookings WHERE id = 15;	S2 cũng đọc booking #15 (không có khóa)
	-- Kết quả: PENDING	S2 cũng thấy PENDING – cả hai đang đọc cùng một snapshot
-- Bước 2: Lễ tân A xác nhận booking		

<code>UPDATE bookings SET status = 'CONFIRMED' WHERE id = 15;</code>		S1 cập nhật sang CONFIRMED
<code>COMMIT;</code>		S1 commit thành công – booking #15 = CONFIRMED
	-- Bước 3: Lễ tân B hủy booking	
	<code>UPDATE bookings SET status = 'CANCELLED' WHERE id = 15;</code>	S2 vẫn nghĩ booking đang PENDING, tiến hành hủy
	<code>COMMIT;</code>	S2 commit thành công – booking #15 bị ghi đè thành CANCELLED
<code>SELECT status FROM bookings WHERE id = 15;</code>		Kiểm tra lại kết quả cuối
-- Kết quả: CANCELLED (Lost Update)		Thao tác CONFIRMED của S1 bị mất hoàn toàn

Mô tả giải pháp (Khi đã có FOR UPDATE):

Bảng dsfhd: Mô tả giải pháp (Khi đã có FOR UPDATE)

Session 1 (S1) – Lễ tân A	Session 2 (S2) – Lễ tân B	Giải thích
---------------------------	---------------------------	------------

<code>SELECT status, room_id FROM bookings WHERE id = 15 FOR UPDATE;</code>		S1 đọc VÀ khóa dòng booking #15
-- Kết quả: PENDING		S1 giữ exclusive lock trên dòng này
	<code>SELECT status, room_id FROM bookings WHERE id = 15 FOR UPDATE;</code>	S2 cố đọc booking #15 với FOR UPDATE
	Session bị blocking	S2 bị BLOCK – phải chờ S1 giải phóng khóa
<code>UPDATE bookings SET status = 'CONFIRMED' WHERE id = 15;</code>		S1 cập nhật sang CONFIRMED
<code>COMMIT;</code>		S1 commit, khóa được giải phóng
	-- Kết quả: CONFIRMED	S2 được thực thi, đọc lại trạng thái mới nhất = CONFIRMED
		Tùy quy định hệ thống, S2 có thể bị từ chối hoặc tiếp tục

Giải pháp áp dụng:

Các procedure như `sp_update_booking_status`, `sp_process_check_in`, và `sp_process_check_out` đều sử dụng `SELECT ... FOR UPDATE` để khóa dòng booking ngay tại bước đọc trạng thái. Giao tác đến sau phải chờ giao tác đến trước kết thúc rồi mới đọc lại trạng thái thực tế. Kết quả là Lost Update sẽ bị loại bỏ hoàn toàn.

```

SELECT status, room_id
INTO v_current_status, v_room_id
FROM bookings
WHERE id = p_booking_id
FOR UPDATE;

IF v_current_status = p_new_status THEN
    RAISE_APPLICATION_ERROR(-20200, 'Booking đã ở trạng thái: ' || p_new_status);
END IF;

IF v_current_status = 'CANCELLED' THEN
    RAISE_APPLICATION_ERROR(-20201, 'Không thể thay đổi booking đã bị hủy');
END IF;

IF v_current_status = 'CHECKED_OUT' THEN
    RAISE_APPLICATION_ERROR(-20202, 'Không thể thay đổi booking đã trả phòng');
END IF;

UPDATE bookings
SET status = p_new_status
WHERE id = p_booking_id;

```

4.4.3 Non-Repeatable Read – Đọc giá phòng thay đổi giữa chừng

Non-Repeatable Read xảy ra khi một giao tác đọc cùng một dòng dữ liệu hai lần và nhận được kết quả khác nhau, vì một giao tác khác đã UPDATE dòng đó và commit giữa hai lần đọc.

Kịch bản trong hệ thống:

- T=1: Khách A xem chi tiết phòng Deluxe:
 - Query room_types: base_price = 2,000,000 VND/đêm
 - Giao diện hiển thị: "Giá ước tính 3 đêm: 6,000,000 VND"
- T=2: Admin cập nhật seasonal_price cho dịp lễ 30/4:
 - Thêm bản ghi seasonal_prices với price_multiplier = 1.5
 - COMMIT thành công
- T=3: Khách A nhấn "Xác nhận đặt phòng"
 - SP_CREATE_BOOKING gọi FN_CALCULATE_ROOM_PRICE
 - Hàm đọc lại seasonal_prices: tìm thấy hệ số x1.5
 - Giá tính ra: $2,000,000 \times 1.5 \times 3 = 9,000,000$ VND (khác so với giá ban đầu)
 - Booking được tạo với raw_price = 9,000,000 VND

Minh họa Non-Repeatable Read:*Bảng ???: Minh họa Non-Repeatable Read*

Session 1 (S1) – Khách A	Session 2 (S2) – Admin	Giải thích
-- Bước 1: Khách xem giá phòng Deluxe		
<code>SELECT base_price FROM room_types WHERE id = 1;</code>		S1 đọc giá gốc phòng Deluxe (room_type_id = 1)
-- Kết quả: 2000000		Giao diện hiển thị "Giá ước tính 3 đêm: 6,000,000 VND"

	-- Bước 2: Admin thêm giá mùa lễ	
	<pre>INSERT INTO seasonal_prices (room_type_id, name, start_date, end_date, price_multiplier, priority) VALUES (1, 'Lễ 30/4', DATE '2025-04- 28', DATE '2025-05-02', 1.5, 10);</pre>	S2 thêm hệ số giá x1.5 cho dịp lễ 30/4
	COMMIT;	S2 commit thành công
-- Bước 3: Khách xác nhận đặt phòng		
<pre>SELECT base_price FROM room_types WHERE id = 1;</pre>		S1 đọc lại giá gốc – vẫn là 2,000,000
-- Kết quả: 2000000		Đây chưa phải Non-Repeatable Read vì cùng dòng room_types

<code>SELECT price_multiplier FROM seasonal_prices WHERE room_type_id = 1 AND DATE '2025-04-30' BETWEEN start_date AND end_date;</code>		S1 kiểm tra seasonal price cho ngày 30/4
-- Kết quả: 1.5		Đây là Non-Repeatable Read: cùng query nhưng kết quả khác do S2 đã INSERT
Giá thực tế: $2,000,000 \times 1.5 = 3,000,000$ VND/đêm		fn_calculate_room_price tính ra 9,000,000 VND cho 3 đêm, khác so với giá đã được hiển thị

Giải pháp áp dụng: Hệ thống chấp nhận Non-Repeatable Read ở tầng hiển thị, nhưng đảm bảo giá tại thời điểm tạo booking luôn chính xác. SP_CREATE_BOOKING gọi FN_CALCULATE_ROOM_PRICE ngay trước khi INSERT – hàm này đọc lại toàn bộ daily_prices và seasonal_prices theo thứ tự ưu tiên tại thời điểm đặt, không dùng giá đã cache từ lúc hiển thị. Về phía giao diện, hệ thống nên hiển thị cảnh báo "Giá có thể thay đổi cho đến khi xác nhận đặt phòng".

```
-- SP_CREATE_BOOKING: Gọi FN_CALCULATE_ROOM_PRICE ngay trước INSERT
-- đảm bảo giá luôn được tính lại tại thời điểm đặt phòng, không phải thời điểm
hiển thị
v_raw_price := FN_CALCULATE_ROOM_PRICE(p_room_type_id, p_check_in_date,
p_check_out_date);

-- FN_CALCULATE_ROOM_PRICE: Duyệt từng ngày, ưu tiên daily_prices >
seasonal_prices > base_price
-- Ưu tiên 1: Kiểm tra daily_prices (giá đặc biệt từng ngày)
```

```

SELECT price INTO v_daily_price
FROM daily_prices
WHERE room_type_id = p_room_type_id AND price_date = v_current_date;

-- Ưu tiên 2: Nếu không có daily → kiểm tra seasonal_prices (hệ số theo mùa)
SELECT price_multiplier INTO v_multiplier
FROM seasonal_prices
WHERE room_type_id = p_room_type_id
AND v_current_date BETWEEN start_date AND end_date
ORDER BY priority DESC
FETCH FIRST 1 ROW ONLY;

v_day_price := v_base_price * v_multiplier;

-- Ưu tiên 3: Nếu không có seasonal → dùng base_price từ room_types
v_day_price := v_base_price;

```

4.4.4 Deadlock – Hai booking cùng khóa hai phòng theo thứ tự ngược nhau

Deadlock là tình trạng hai hoặc nhiều tiến trình Transactions bị treo vô thời hạn vì mỗi tiến trình đều đang giữ tài nguyên mà tiến trình kia cần, tạo thành một vòng lặp chờ đợi.

Kịch bản trong hệ thống: Tình huống xảy ra khi hệ thống nhận đồng thời hai yêu cầu đặt nhiều phòng, và thứ tự khóa ngược nhau.

- Giao tác A – Khách đoàn cần phòng 101 và 102:
 - T=1: FOR UPDATE phòng 101, khóa thành công
 - T=2: FOR UPDATE phòng 102, CHỜ (B đang giữ), blocking
- Giao tác B – Khách đoàn khác cần phòng 102 và 101:
 - T=1: FOR UPDATE phòng 102, khóa thành công
 - T=2: FOR UPDATE phòng 101, CHỜ (A đang giữ), blocking

A chờ B giải phóng 102, B chờ A giải phóng 101, DEADLOCK sẽ xảy ra. Khi Oracle phát hiện, hệ thống sẽ trả về kết quả ORA-00060: deadlock detected while waiting for resource, kèm theo đó là một giao tác bị rollback câu lệnh hiện tại.

Mô tả Deadlock (khi chưa có SKIP LOCKED):*Bảng abasd: Mô tả Deadlock (khi chưa có SKIP LOCKED)*

Session 1 (S1) – Lễ tân A	Session 2 (S2) – Lễ tân B	Giải thích
-- Bước 1: Khóa phòng 101		
SELECT id FROM rooms WHERE id = 101 FOR UPDATE;		S1 khóa phòng 101 thành công
Khóa thành công		S1 đang giữ exclusive lock trên phòng 101
	-- Bước 2: Khóa phòng 102	
	SELECT id FROM rooms WHERE id = 102 FOR UPDATE;	S2 khóa phòng 102 thành công
	Khóa thành công	S2 đang giữ exclusive lock trên phòng 102
-- Bước 3: Cố khóa phòng 102		
SELECT id FROM rooms WHERE id = 102 FOR UPDATE;		S1 cố khóa phòng 102 – đang bị S2 giữ

		S1 bị BLOCK, chờ S2 giải phóng phòng 102
	SELECT id FROM rooms WHERE id = 101 FOR UPDATE;	S2 cố khóa phòng 101 – đang bị S1 giữ
		S2 bị BLOCK, chờ S1 giải phóng phòng 101
	-- ORA-00060: deadlock detected	Oracle phát hiện vòng tròn chờ, chọn S2 làm nạn nhân, rollback câu lệnh của S2
-- S1 được tiếp tục thực thi		S1 nhận được khóa phòng 102 sau khi S2 bị rollback

Mô tả Deadlock (Sau khi áp dụng SKIP LOCKED)

Bảng absc: Mô tả Deadlock (sau khi áp dụng SKIP LOCKED)

Session 1 (S1) – sp_create_booking	Session 2 (S2) – sp_create_booking	Giải thích
<pre>SELECT r.id INTO v_room_id FROM rooms r WHERE r.room_type_id = p_room_type_id AND r.status != 'MAINTENANCE' AND NOT EXISTS (SELECT 1 FROM bookings b WHERE b.room_id = r.id AND b.status IN ('PENDING','CONFIRMED','CHECKED_IN') AND p_check_in_date < b.check_out_date AND p_check_out_date > b.check_in_date) ORDER BY r.floor, r.room_number FETCH FIRST 1 ROW ONLY FOR UPDATE SKIP LOCKED;</pre>		S1 chạy SP_CREATE_BOOKING – tìm phòng trống đầu tiên và khóa ngay
-- Kết quả: room_id = 101		S1 khóa phòng 101 thành công
	<pre>SELECT r.id INTO v_room_id FROM rooms r WHERE r.room_type_id = p_room_type_id AND r.status != 'MAINTENANCE' AND NOT EXISTS (SELECT 1 FROM bookings b WHERE b.room_id = r.id AND b.status IN ('PENDING','CONFIRMED','CHECKED_IN') AND p_check_in_date < b.check_out_date AND p_check_out_date ></pre>	S2 cũng chạy SP_CREATE_BOOKING – tìm phòng trống

	b.check_in_date) ORDER BY r.floor, r.room_number FETCH FIRST 1 ROW ONLY FOR UPDATE SKIP LOCKED;	
	-- Kết quả: room_id = 102	S2 thấy phòng 101 đang bị S1 khóa → bỏ qua → chọn phòng 102. Không xảy ra Deadlock.
INSERT INTO bookings (...) VALUES (...);		S1 tạo booking cho phòng 101
COMMIT;		S1 hoàn tất, giải phóng khóa phòng 101
	INSERT INTO bookings (...) VALUES (...);	S2 tạo booking cho phòng 102
	COMMIT;	S2 hoàn tất – hai booking song song, không xung đột

Giải pháp áp dụng: Trong phạm vi đồ án môn học, procedure `sp_create_booking` chỉ tìm một phòng cho mỗi lần đặt, và dùng `SKIP LOCKED` để bỏ qua phòng đang bị khóa. Bằng cách kết hợp `ORDER BY r.floor, r.room_number` (thứ

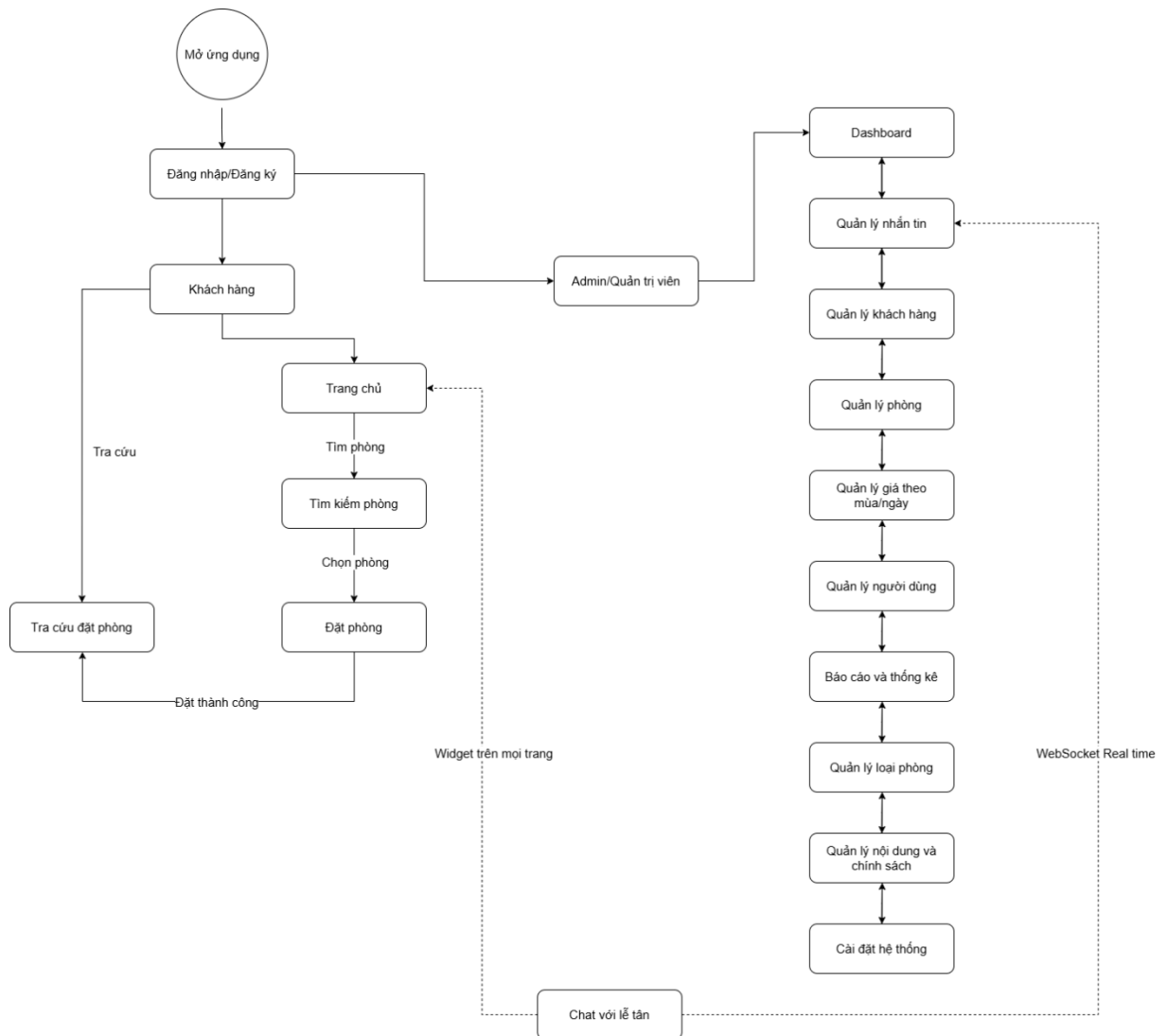
tự ưu tiên nhất quán) và SKIP LOCKED, điều kiện hình thành deadlock bị phá vỡ hoàn toàn: không có giao tác nào giữ một khóa trong khi chờ khóa khác.

```
SELECT r.id INTO v_room_id
FROM rooms r
WHERE r.room_type_id = p_room_type_id
  AND r.status != 'MAINTENANCE'
  AND NOT EXISTS ( ... )
ORDER BY r.floor, r.room_number
FETCH FIRST 1 ROW ONLY
FOR UPDATE SKIP LOCKED;
```

Chương 5: THIẾT KẾ GIAO DIỆN

5.1 Sơ đồ liên kết các màn hình

Dưới đây là sơ đồ liên kết các màn hình của hệ thống quản lý khách sạn:



Hình 5.1: Sơ đồ liên kết các màn hình của hệ thống quản lý khách sạn

5.2 Danh sách các màn hình

Bảng 5.1: Danh sách các màn hình

STT	Màn hình	Loại màn hình	Chức năng
1	Màn hình Đăng nhập / Đăng ký	Màn hình nhập liệu	Cho phép người dùng đăng nhập bằng tài khoản đã có hoặc đăng ký tài khoản mới. Hệ thống tự động điều hướng đến giao diện Client hoặc Admin tùy theo vai

			trò (Client, Admin, Manager, Receptionist).
2	Màn hình trang chủ	Màn hình chính	Hiển thị tổng quan về khách sạn: video giới thiệu, danh sách tiện nghi, loại phòng nổi bật, câu hỏi thường gặp và đánh giá từ khách hàng. Cung cấp nút điều hướng đến Tìm phòng và Tra cứu đặt phòng.
3	Màn hình Tìm kiếm phòng	Màn hình tra cứu	Cho phép khách hàng tìm kiếm phòng trống theo ngày check-in/check-out, loại phòng, số lượng khách. Hiển thị danh sách kết quả kèm giá, ảnh minh họa và tiện nghi từng phòng.
4	Màn hình Đặt phòng	Màn hình nhập liệu	Cho phép khách hàng tạo booking: chọn phòng, nhập thông tin cá nhân, áp dụng mã khuyến mãi. Hiển thị tổng giá và xác nhận đặt phòng.
5	Màn hình Tra cứu đặt phòng	Màn hình tra cứu	Hiển thị thông tin chi tiết booking đã đặt: trạng thái đặt phòng, thông tin phòng, giá tiền, ngày check-in/check-out. Cho phép khách hàng theo dõi tình trạng đặt phòng.
6	Màn hình Chat với lễ tân	Màn hình tương tác	Widget chat real-time qua WebSocket (STOMP), hiển thị dạng popup ở góc phải dưới trên mọi trang Client. Cho phép khách hàng gửi/nhận tin nhắn trực tiếp với lễ tân hoặc admin.

7	Màn hình Dashboard	Màn hình chính	Hiển thị tổng quan hệ thống quản trị: tổng doanh thu, số booking mới, tỷ lệ phòng trống, biểu đồ thống kê theo thời gian (ngày/tuần/tháng/năm). Là điểm trung tâm điều hướng đến các màn hình quản lý khác.
8	Màn hình Quản lý nhắn tin	Màn hình quản lý	Hiển thị danh sách cuộc hội thoại với khách hàng qua WebSocket real-time. Admin/Lễ tân có thể xem lịch sử, trả lời tin nhắn và quản lý các cuộc trò chuyện.
9	Màn hình Quản lý khách hàng	Màn hình quản lý	Quản lý thông tin khách hàng (CRUD): thêm, sửa, xóa, xem chi tiết. Hiển thị danh sách khách hàng với thông tin liên hệ, trạng thái VIP, quốc tịch, lịch sử đặt phòng.
10	Màn hình Quản lý phòng	Màn hình quản lý	Quản lý danh sách phòng (CRUD): thêm, sửa, xóa phòng. Cập nhật trạng thái phòng (Available, Occupied, Maintenance, Reserved). Lọc theo tầng, trạng thái, loại phòng.
11	Màn hình Quản lý giá theo mùa/ngày – Quản lý khuyến mãi	Màn hình quản lý	Quản lý bảng giá phòng theo mùa (Seasonal Pricing) với hệ số nhân giá, giá đặc biệt theo ngày cụ thể (Daily Pricing), và quản lý mã khuyến mãi (Promotions).
12	Màn hình Quản lý người dùng	Màn hình quản lý	Quản lý tài khoản người dùng hệ thống (CRUD). Phân quyền theo

			vai trò: SUPERADMIN, ADMIN, MANAGER, RECEPTIONIST. Quản lý trạng thái tài khoản (active, inactive, suspended).
13	Màn hình Báo cáo và thống kê	Màn hình quản lý	Hiển thị báo cáo doanh thu theo tháng/quý/năm bằng biểu đồ (Recharts). Thống kê số booking, tỷ lệ lấp đầy phòng, doanh thu theo loại phòng. Hỗ trợ xuất báo cáo.
14	Màn hình Quản lý loại phòng	Màn hình quản lý	Quản lý danh sách loại phòng (Room Types): thêm, sửa, xóa loại phòng. Quản lý giá cơ bản, sức chứa tối đa, ảnh minh họa, danh sách tiện nghi theo từng loại.
15	Màn hình Quản lý nội dung và chính sách	Màn hình quản lý	Quản lý chính sách khách sạn (hủy phòng, hoàn tiền), câu hỏi thường gặp (FAQ), đánh giá từ khách hàng. CRUD nội dung hiển thị trên trang Client.
16	Màn hình Cài đặt hệ thống	Màn hình cấu hình	Cấu hình tài khoản cá nhân (thông tin profile, đổi mật khẩu), giao diện hiển thị (theme sáng/tối), tùy chọn thông báo và các thiết lập chung của hệ thống.

5.3 Mô tả chi tiết các màn hình

5.3.1 Màn hình đăng nhập/đăng ký

5.3.1.1. Giao diện màn hình

a. Giao diện màn hình đăng nhập

Lumina Suites

LUMINA SUITES®

(11) (12) (13) (14)

(1) ☰ Lumina Suites

Đăng nhập (2)

Nhập email và mật khẩu để đăng nhập vào hệ thống khách sạn (3)

Email

name@example.com (4)

Mật khẩu (5) (7) Quên mật khẩu?

Đăng nhập (8)

HOẶC

Chưa có tài khoản? Đăng ký (9)

(10) [User Profile Icon]

Hình 5.2: Giao diện màn hình đăng nhập

b. Giao diện màn hình đăng ký

Lumina Suites

LUMINA SUITES®

(13) (14) (15) (16)

(1) ☰ Lumina Suites

Tạo tài khoản (2)

Nhập thông tin để đăng ký tài khoản mới (3)

Họ tên

Nguyễn Văn A (4)

Email

name@example.com (5)

Mật khẩu (6) (7)

Xác nhận mật khẩu (8) (9)

Đăng ký (10)

HOẶC

Đã có tài khoản? Đăng nhập (11)

(12) [User Profile Icon]

Hình 5.3: Giao diện màn hình đăng ký

5.3.1.2. Mô tả các đối tượng trên màn hình

a. Màn hình đăng nhập

Bảng 5.2: Mô tả các đối tượng trên màn hình đăng nhập

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	lblLogo	Label		Hiển thị logo và tên "Lumina Suites"
2	lblTitle	Label		Hiển thị tiêu đề "Đăng nhập"
3	lblSubtitle	Label		Hiển thị hướng dẫn "Nhập email và mật khẩu để đăng nhập vào hệ thống khách sạn"
4	txtEmail	TextBox	Bắt buộc, định dạng email	Nhập địa chỉ email đăng nhập
5	txtPassword	TextBox (Password)	Bắt buộc	Nhập mật khẩu đăng nhập
6	btnTogglePassword	Button (Icon)		Hiện/ẩn mật khẩu đã nhập
7	lnkForgotPassword	HyperLink		Điều hướng đến trang quên mật khẩu
8	btnLogin	Button		Xử lý đăng nhập vào hệ thống
9	lnkRegister	HyperLink		Điều hướng đến trang đăng ký tài khoản
10	btnChat	Button (Floating)		Mở widget chat với lễ tân
11	btnThemeToggle	Button (Icon)		Chuyển đổi giao diện sáng/tối
12	btnSearch	Button (Icon)		Mở chức năng tìm kiếm phòng

13	btnBookingLookup	Button (Icon)		Điều hướng đến tra cứu đặt phòng
14	btnUserProfile	Button (Icon)		Điều hướng đến đăng nhập / hồ sơ cá nhân

b. Màn hình đăng ký

Bảng 5.3: Mô tả các đối tượng trên màn hình đăng ký

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	lblLogo	Label		Hiển thị logo và tên "Lumina Suites"
2	lblTitle	Label		Hiển thị tiêu đề "Tạo tài khoản"
3	lblSubtitle	Label		Hiển thị hướng dẫn "Nhập thông tin để đăng ký tài khoản mới"
4	txtFullName	TextBox	Bắt buộc	Nhập họ tên người dùng
5	txtEmail	TextBox	Bắt buộc, định dạng email	Nhập địa chỉ email đăng ký
6	txtPassword	TextBox (Password)	Bắt buộc, tối thiểu 6 ký tự	Nhập mật khẩu
7	btnTogglePassword1	Button (Icon)		Hiện/ẩn mật khẩu
8	txtConfirmPassword	TextBox (Password)	Bắt buộc, phải trùng mật khẩu	Xác nhận lại mật khẩu
9	btnTogglePassword2	Button (Icon)		Hiện/ẩn mật khẩu xác nhận
10	btnRegister	Button		Xử lý đăng ký tài khoản mới

11	lnkLogin	Hyperlink		Điều hướng về trang đăng nhập
12	btnChat	Button (Floating)		Mở widget chat với lễ tân
13	btnThemeToggle	Button (Icon)		Chuyển đổi giao diện sáng/tối
14	btnSearch	Button (Icon)		Mở chức năng tìm kiếm phòng
15	btnBookingLookup	Button (Icon)		Điều hướng đến tra cứu đặt phòng
16	btnUserProfile	Button (Icon)		Điều hướng đến đăng nhập / hồ sơ cá nhân

5.3.1.3. Danh sách biến cố và xử lý tương ứng trên màn hình

a. Màn hình đăng nhập

Bảng 5.4: Danh sách biến cố và xử lý tương ứng trên màn hình đăng nhập

STT	Biến cố	Xử lý	Ghi chú
1	btnLogin	Kiểm tra email và mật khẩu, nếu hợp lệ thì đăng nhập và điều hướng đến trang chủ (Client) hoặc Dashboard (Admin). Nếu sai, hiển thị thông báo lỗi.	Gọi API POST /auth/login
2	btnTogglePassword	Chuyển đổi hiển thị/ẩn nội dung mật khẩu trong ô txtPassword.	
3	lnkForgotPassword	Di chuyển đến màn hình quên mật khẩu, hiển thị form nhập email để đặt lại mật khẩu.	
4	lnkRegister	Di chuyển đến màn hình Đăng ký tài khoản, hiển thị form đăng ký.	

5	btnChat	Mở widget chat nổi (popup) ở góc phải dưới, kết nối WebSocket đến server để nhận tin real-time với lễ tân.	
---	---------	--	--

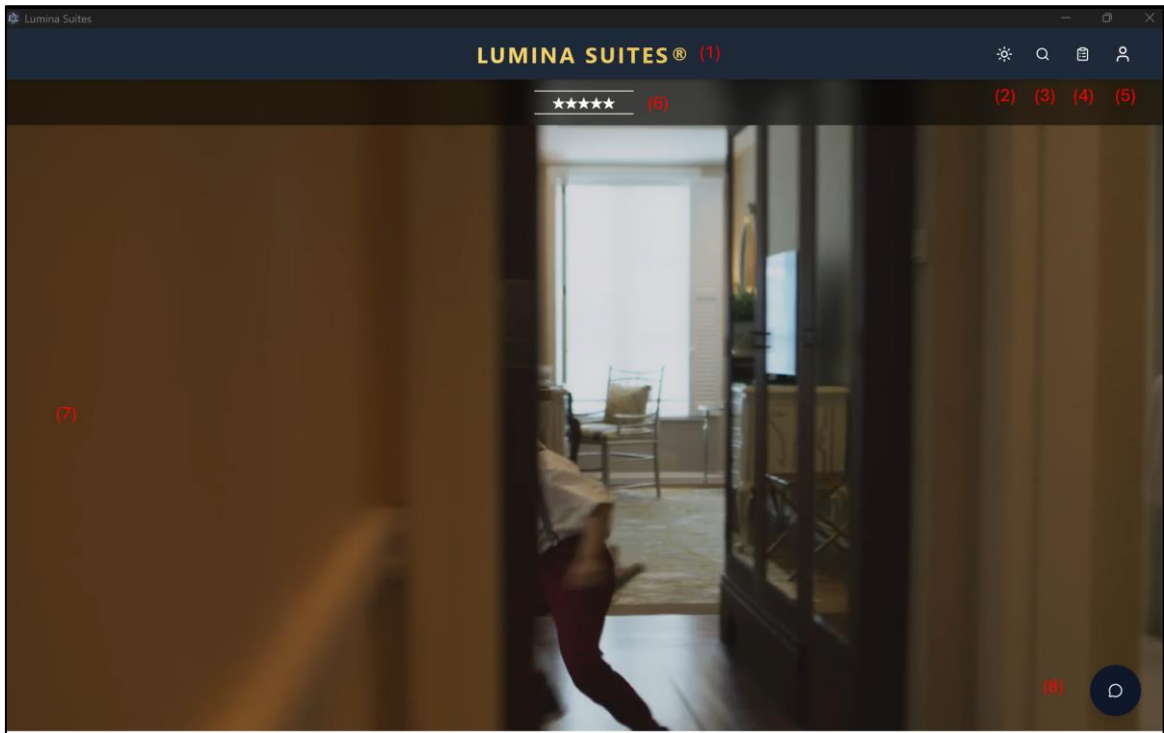
b. Màn hình đăng ký

Bảng 5.5: Danh sách biến cố và xử lý tương ứng trên màn hình đăng ký

STT	Biến cố	Xử lý	Ghi chú
1	btnRegister	zKiểm tra tính hợp lệ của các trường (họ tên, email, mật khẩu, xác nhận mật khẩu). Nếu hợp lệ, tạo tài khoản mới và điều hướng về màn hình Đăng nhập. Nếu sai, hiển thị thông báo lỗi.	Gọi API POST /auth/register
2	btnTogglePassword1	Chuyển đổi hiển thị/ẩn nội dung mật khẩu trong ô txtPassword.	
3	btnTogglePassword2	Chuyển đổi hiển thị/ẩn nội dung mật khẩu trong ô txtConfirmPassword.	
4	lnkLogin	Di chuyển về màn hình Đăng nhập, hiển thị form đăng nhập.	
5	btnChat	Mở widget chat nổi (popup) để nhận tin real-time với lễ tân.	

5.3.2 Màn hình trang chủ

a. Giao diện màn hình



Hình 5.4: Giao diện màn hình chính

b. Mô tả các đối tượng trên màn hình

Bảng 5.6: Mô tả các đối tượng trên màn hình trang chủ

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	navBar	NavigationBar		Thanh điều hướng chính với logo "LUMINA SUITES®"
2	btnThemeToggle	Button (Icon)		Chuyển đổi giao diện sáng/tối
3	btnSearch	Button (Icon)		Mở chức năng tìm kiếm phòng
4	btnBookingLookup	Button (Icon)		Điều hướng đến tra cứu đặt phòng
5	btnUserProfile	Button (Icon)		Điều hướng đến đăng nhập / hồ sơ cá nhân

6	lblStarRating	Label		Hiển thị xếp hạng 5 sao của khách sạn
7	carVideoIntro	Carousel		Hiển thị video giới thiệu khách sạn
8	btnChat	Button (Floating)		Mở widget chat với lễ tân

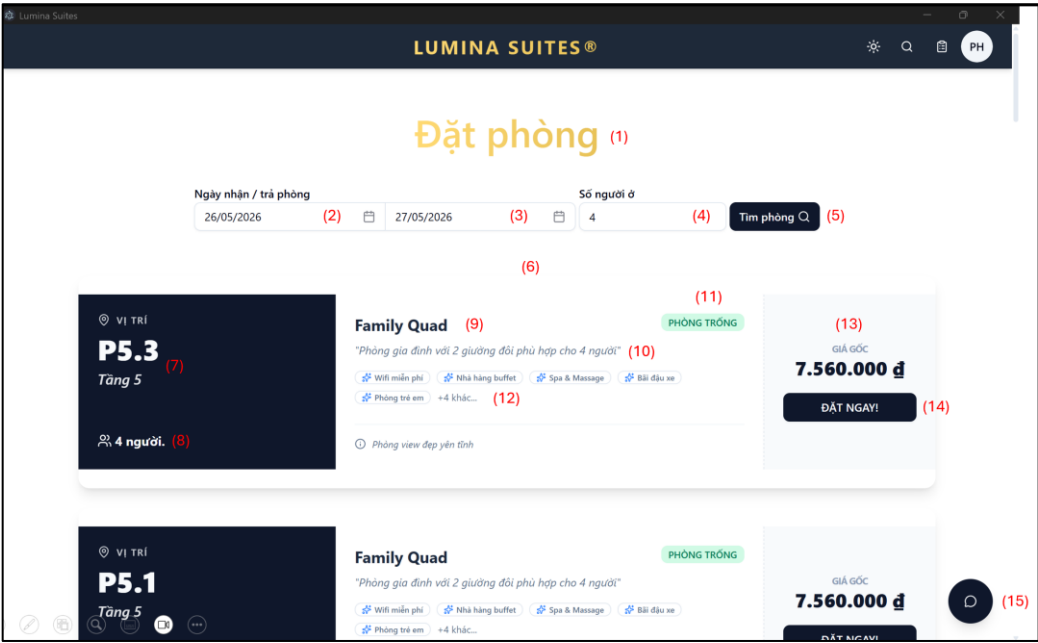
c. Danh sách biến cố và xử lý tương ứng

Bảng 5.7: Danh sách biến cố và xử lý tương ứng trên màn hình trang chủ

STT	Biến cố	Xử lý	Ghi chú
1	btnThemeToggle	Chuyển đổi giao diện giữa chế độ sáng (Light) và tối (Dark). Lưu tùy chọn vào localStorage.	
2	btnSearch	Mở màn hình Tìm kiếm & Đặt phòng, hiển thị form tìm phòng trống.	
3	btnBookingLookup	Di chuyển đến màn hình Tra cứu đặt phòng, hiển thị thông tin booking của khách hàng.	
4	btnUserProfile	Nếu chưa đăng nhập: di chuyển đến màn hình Đăng nhập. Nếu đã đăng nhập: hiển thị menu hồ sơ cá nhân.	
5	btnChat	Mở widget chat nổi (popup) để nhắn tin real-time với lễ tân.	

5.3.3. Màn hình Tìm kiếm & Đặt phòng

a. Giao diện màn hình



Hình 5.5: Giao diện tìm kiếm và đặt phòng

b. Mô tả các đối tượng trên màn hình

Bảng 5.8: Mô tả các đối tượng trên màn hình tìm kiếm và đặt phòng

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	lblTitle	Label		Hiển thị tiêu đề "Đặt phòng"
2	dtpCheckIn	DatePicker	Bắt buộc, lớn hơn hoặc bằng ngày hiện tại	Chọn ngày nhận phòng
3	dtpCheckOut	DatePicker	Bắt buộc, lớn hơn ngày nhận phòng	Chọn ngày trả phòng
4	txtGuests	TextBox (Number)	Bắt buộc, lớn hơn hoặc bằng 1	Nhập số người ở
5	btnSearch	Button		Tìm phòng trống theo điều kiện

6	lstRoomResults	ListView		Danh sách phòng trống tìm được
7	lblRoomLocation	Label		Hiển thị vị trí phòng (số phòng, tầng)
8	lblRoomCapacity	Label		Hiển thị sức chứa phòng
9	lblRoomType	Label		Hiển thị tên loại phòng (Family Quad, ...)
10	lblRoomDescription	Label		Mô tả ngắn về loại phòng
11	lblRoomStatus	Label (Badge)		Trạng thái phòng (PHÒNG TRỐNG)
12	lstAmenities	TagList		Danh sách tiện nghi phòng (Wifi miễn phí, Spa, ...)
13	lblPrice	Label		Hiển thị giá gốc phòng
14	btnBookNow	Button		Đặt phòng được chọn ngay
15	btnChat	Button (Floating)		Mở widget chat với lễ tân

c. Danh sách biến cố và xử lý tương ứng

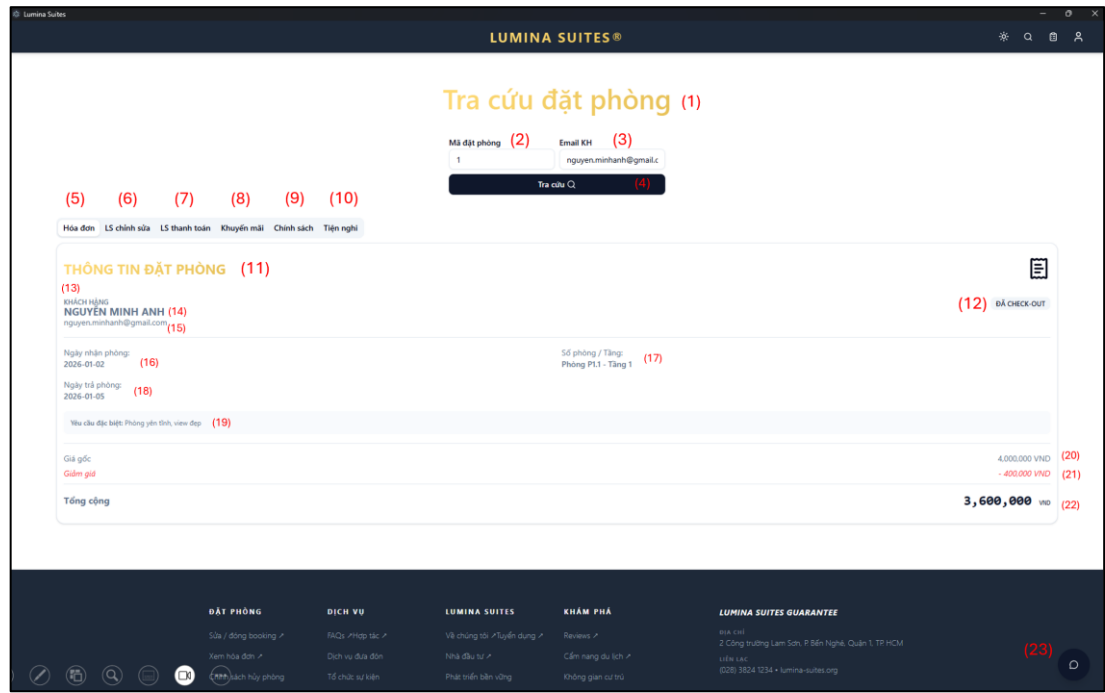
Bảng 5.9: Danh sách biến cố và xử lý tương ứng trên màn hình tìm kiếm và đặt phòng

STT	Biến cố	Xử lý	Ghi chú
1	btnSearch	Gửi yêu cầu tìm phòng trống theo ngày check-in, check-out và số người. Kết quả hiển thị trong lstRoomResults.	Gọi API GET /rooms/available
2	dtpCheckIn	Chọn ngày nhận phòng. Tự động cập nhật dtpCheckOut nếu ngày trả phòng nhỏ hơn ngày nhận.	Ràng buộc lớn hơn hoặc bằng ngày hiện tại

3	ctpCheckOut	Chọn ngày trả phòng. Kiểm tra phải lớn hơn ngày nhận phòng.	
4	btnBookNow	Tạo booking cho phòng được chọn. Hiện thị form xác nhận thông tin đặt phòng (tên, email, yêu cầu đặc biệt, mã khuyến mãi).	Gọi API POST /bookings
5	btnChat	Mở widget chat nổi (popup) để nhắn tin real-time với lễ tân.	

5.3.4 Màn hình tra cứu đặt phòng

a. Giao diện màn hình



Hình 5.6: Giao diện màn hình tra cứu đặt phòng

b. Mô tả các đối tượng trên màn hình

Bảng 5.10: Mô tả các đối tượng trên màn hình tra cứu đặt phòng

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	lblTitle	Label		Tiêu đề "Tra cứu đặt phòng"
2	txtBookingId	TextBox	Bắt buộc	Nhập mã đặt phòng cần tra cứu

3	txtEmailKH	TextBox	Bắt buộc, định dạng email	Nhập email khách hàng để tra cứu
4	btnLookup	Button		Thực hiện tra cứu đặt phòng theo mã và email
5	tabInvoice	Tab		Tab "Hóa đơn" – xem thông tin đặt phòng
6	tabEditHistory	Tab		Tab "LS chỉnh sửa" – xem lịch sử chỉnh sửa booking
7	tabPaymentHistory	Tab		Tab "LS thanh toán" – xem lịch sử thanh toán
8	tabPromotions	Tab		Tab "Khuyến mãi" – xem mã giảm giá đã áp dụng
9	tabPolicies	Tab		Tab "Chính sách" – xem chính sách áp dụng
10	tabAmenities	Tab		Tab "Tiện nghi" – xem tiện nghi phòng
11	lblBookingTitle	Label		Tiêu đề "THÔNG TIN ĐẶT PHÒNG"
12	lblStatus	Label (Badge)		Trạng thái booking (ĐÃ CHECK-OUT)
13	lblCustomerLabel	Label		Nhãn "KHÁCH HÀNG"
14	lblCustomerName	Label		Hiển thị tên khách hàng
15	lblCustomerEmail	Label		Hiển thị email khách hàng
16	lblCheckInDate	Label		Hiển thị ngày nhận phòng (2026-01-02)

17	lblRoomInfo	Label		Hiển thị số phòng / tầng (Phòng P11 - Tầng 1)
18	lblCheckOutDate	Label		Hiển thị ngày trả phòng (2026-01-05)
19	lblSpecialRequest	Label		Hiển thị yêu cầu đặc biệt (Phòng yên tĩnh, view đẹp)
20	lblOriginalPrice	Label		Hiển thị giá gốc (4,000,000 VNĐ)
21	lblDiscount	Label		Hiển thị số tiền giảm giá (-400,000 VNĐ)
22	lblTotalPrice	Label		Hiển thị tổng cộng thanh toán (3,600,000 VNĐ)
23	btnChat	Button (Floating)		Mở widget chat với lễ tân

c. Danh sách các biến cố và xử lý tương ứng

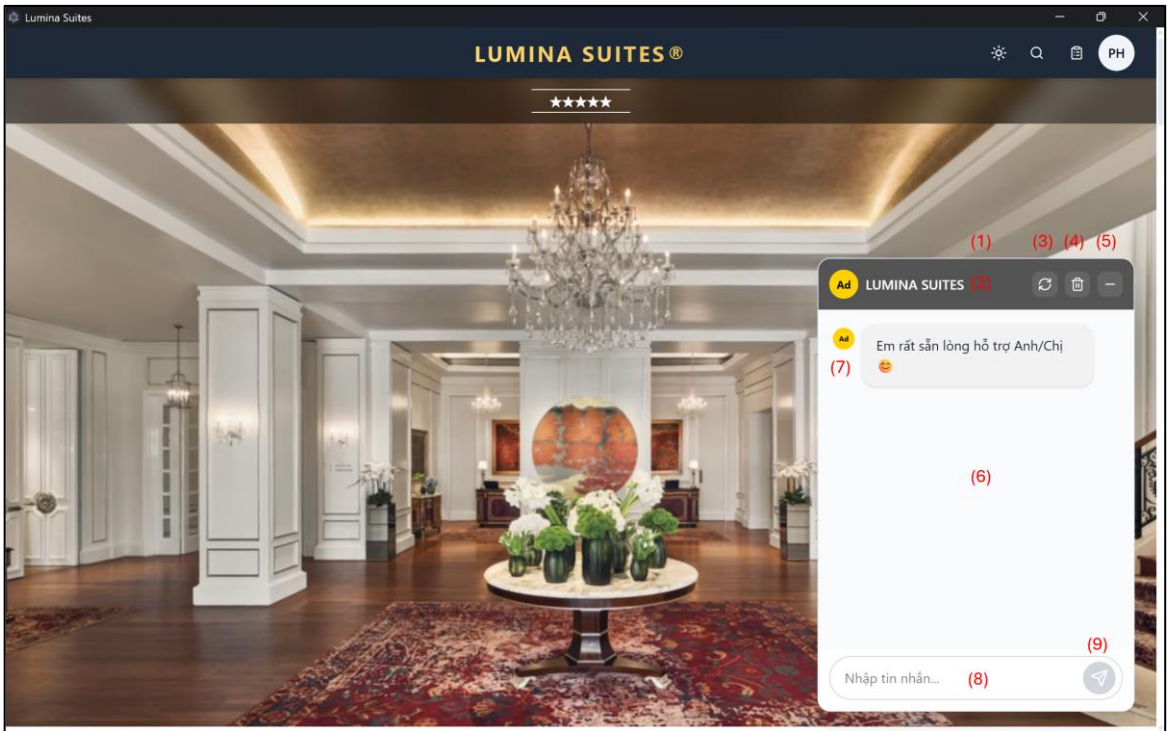
Bảng 5.11: Danh sách biến cố và xử lý tương ứng trên màn hình tra cứu đặt phòng

STT	Biến cố	Xử lý	Ghi chú
1	btnLookup	Tra cứu thông tin đặt phòng theo mã booking (txtBookingId) và email khách hàng (txtEmailKH). Hiển thị chi tiết booking trong card thông tin và các tab bên dưới.	Gọi API GET /bookings/{id}
2	tabInvoice	Chuyển sang tab Hóa đơn, hiển thị thông tin chi tiết đặt phòng và thanh toán.	

3	tabEditHistory	Chuyển sang tab LS chỉnh sửa, hiển thị danh sách các lần thay đổi booking.	
4	tabPaymentHistory	Chuyển sang tab LS thanh toán, hiển thị các giao dịch thanh toán đã thực hiện.	
5	tabPromotions	Chuyển sang tab Khuyến mãi, hiển thị các mã giảm giá đã áp dụng cho booking	
6	tabPolicies	Chuyển sang tab Chính sách, hiển thị các chính sách áp dụng cho booking.	
7	tabAmenities	Chuyển sang tab Tiện nghi, hiển thị danh sách tiện nghi của phòng đã đặt.	
8	btnChat	Mở widget chat nổi (popup) để nhắn tin real-time với lễ tân.	

5.3.5 Màn hình chat với lễ tân

a. Giao diện màn hình



Hình 5.7: Giao diện màn hình chat với lễ tân

b. Mô tả các đối tượng trên màn hình

Bảng 5.12: Mô tả các đối tượng trên màn hình chat với lễ tân

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	pnlChatPopup	Panel (Popup)		Khung chat nổi ở góc phải dưới màn hình
2	lblChatHeader	Label		Hiển thị tiêu đề "LUMINA SUITES" với avatar
3	btnRefresh	Button (Icon)		Làm mới cuộc hội thoại
4	btnDelete	Button (Icon)		Xóa cuộc hội thoại
5	btnMinimize	Button (Icon)		Thu nhỏ khung chat
6	pnlMessages	Panel (Scrollable)		Vùng hiển thị tin nhắn đã gửi/nhận
7	lblMessageBubble	Label (Bubble)		Bong bóng tin nhắn (phân biệt gửi/nhận)

8	txtMessageInput	TextBox		Nhập nội dung tin nhắn
9	btnSend	Button (Icon)		Gửi tin nhắn đến admin/lễ tân

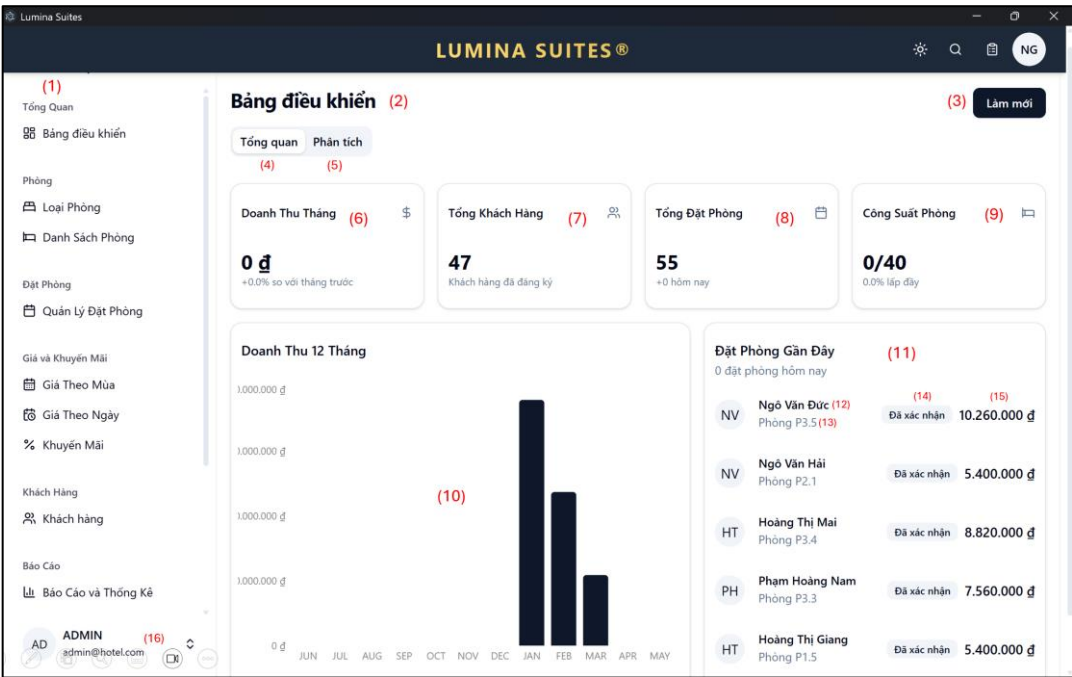
c. Danh sách các biến cố và xử lý tương ứng

Bảng 5.13: Danh sách các biến cố và xử lý tương ứng trên màn hình chat với lễ tân

STT	Biến cố	Xử lý	Ghi chú
1	btnSend	Gửi nội dung tin nhắn trong txtMessageInput đến admin/lễ tân qua WebSocket (STOMP). Tin nhắn xuất hiện ngay trong pnlMessages.	Real-time qua WebSocket
2	btnRefresh	Làm mới lịch sử hội thoại, tải lại danh sách tin nhắn từ server.	
3	btnDelete	Xóa toàn bộ cuộc hội thoại hiện tại sau khi xác nhận từ người dùng.	Hiện thị dialog xác nhận
4	btnMinimize	Thu nhỏ khung chat popup về dạng icon nổi ở góc phải dưới.	
5	txtMessageInput (Enter)	Nhấn phím Enter trong ô nhập tin nhắn sẽ tự động gửi tin nhắn (tương đương nhấn btnSend).	

5.3.6 Màn hình Dashboard (phía Admin)

a. Giao diện màn hình



Hình 5.8: Giao diện màn hình Dashboard

b. Mô tả các đối tượng trên màn hình

Bảng 5.14: Mô tả các đối tượng trên màn hình Dashboard

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	navSidebar	NavigationPanel		Menu điều hướng bên trái
2	lblDashboardTitle	Label		Tiêu đề "Bảng điều khiển"
3	btnRefresh	Button		Làm mới dữ liệu dashboard
4	tabOverview	Tab		Tab "Tổng quan" hiển thị tổng hợp
5	tabAnalysis	Tab		Tab "Phân tích" hiển thị chi tiết
6	cardRevenue	Card		Thẻ hiển thị doanh thu tháng

7	cardTotalCustomers	Card		Thẻ hiển thị tổng khách hàng
8	cardTotalBookings	Card		Thẻ hiển thị tổng đặt phòng
9	cardOccupancy	Card		Thẻ hiển thị công suất phòng
10	chtRevenue12Months	BarChart		Biểu đồ cột doanh thu 12 tháng
11	lstRecentBookings	ListView		Danh sách đặt phòng gần đây
12	lblBookingName	Label		Tên khách hàng trong booking gần đây
13	lblBookingRoom	Label		Số phòng đã đặt
14	lblBookingStatus	Label (Badge)		Trạng thái booking (đã xác nhận)
15	lblBookingPrice	Label		Số tiền booking
16	lblAdminInfo	Label		Thông tin admin đang đăng nhập

c. Danh sách các biến cố và xử lý tương ứng

Bảng 5.15: Danh sách các biến cố và xử lý tương ứng trên màn hình Dashboard

STT	Biến cố	Xử lý	Ghi chú
1	btnRefresh	Tải lại toàn bộ dữ liệu dashboard (doanh thu, booking, công suất) từ server.	Gọi API GET /dashboard
2	tabOverview	Chuyển sang tab Tổng quan, hiển thị các thẻ thống kê tổng hợp và biểu đồ doanh thu.	

3	tabAnalysis	Chuyển sang tab Phân tích, hiển thị biểu đồ chi tiết và so sánh dữ liệu theo kỳ.	
4	navSidebar	Điều hướng đến các màn hình quản lý khác (Phòng, Đặt phòng, Khách hàng, ...) khi nhấn vào các mục menu.	
5	lstRecentBookings (click row)	Nhấn vào một dòng booking gần đây sẽ mở chi tiết booking tương ứng.	

5.3.7 Màn hình quản lý loại phòng

a. Giao diện màn hình



Hình 5.9: Giao diện màn hình quản lý loại phòng

b. Mô tả các đối tượng trên màn hình

Bảng 5.16: Mô tả các đối tượng trên màn hình quản lý loại phòng

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	navSidebar	NavigationPanel		Menu điều hướng bên trái
2	lblTitle	Label		Tiêu đề "Quản lý Loại Phòng"

3	btnAddRoomType	Button		Mở form thêm loại phòng mới
4	txtSearch	TextBox		Tìm kiếm loại phòng theo tên
5	tblRoomTypes	Table		Bảng danh sách loại phòng
6	colRoomTypeName	Column		Cột tên loại phòng
7	colCapacity	Column		Cột sức chứa (số người)
8	colBasePrice	Column		Cột giá cơ bản
9	colDescription	Column		Cột mô tả loại phòng
10	btnActions	Button (Icon)		Menu thao tác (sửa, xóa) cho từng dòng
11	lblPagination	Label		Hiển thị thông tin phân trang
12	btnPrevPage	Button		Chuyển trang trước
13	btnNextPage	Button		Chuyển trang sau

c. Danh sách các biến cố và xử lý tương ứng

Bảng 5.17: Danh sách các biến cố và xử lý tương ứng - quản lý loại phòng

STT	Biến cố	Xử lý	Ghi chú
1	btnAddRoomType	Mở dialog thêm loại phòng mới. Nhập tên, sức chứa, giá cơ bản, mô tả, ảnh và tiện nghi. Lưu thành công sẽ cập nhật lại bảng.	Gọi API POST /room-types
2	txtSearch	Lọc danh sách loại phòng theo từ khóa nhập vào (tìm theo tên loại phòng). Kết quả cập nhật real-time trong bảng.	

3	btnActions (Sửa)	Mở dialog chỉnh sửa thông tin loại phòng được chọn. Cho phép cập nhật tên, giá, sức chứa, mô tả.	Gọi API PUT /room-types/{id}
4	btnActions (Xóa)	Hiện thị dialog xác nhận xóa loại phòng. Nếu xác nhận, xóa loại phòng và cập nhật lại bảng.	Gọi API DELETE /room-types/{id}
5	btnPrevPage	Chuyển về trang trước trong danh sách phân trang. Nếu đang ở trang 1 thì vô hiệu hóa.	
6	btnNextPage	Chuyển sang trang tiếp theo trong danh sách phân trang. Nếu đang ở trang cuối thì vô hiệu hóa.	

5.3.8 Màn hình quản lý phòng

a. Giao diện màn hình



Hình 5.10: Giao diện màn hình quản lý phòng

b. Mô tả các đối tượng trên màn hình

Bảng 5.18: Mô tả các đối tượng trên màn hình quản lý phòng

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	navSidebar	NavigationPanel		Menu điều hướng bên trái
2	lblTitle	Label		Tiêu đề "Quản lý Phòng"
3	lblSubtitle	Label		Mô tả phụ "Quản lý danh sách phòng khách sạn tại đây"
4	btnBatchAdd	Button		Thêm hàng loạt phòng
5	btnAddRoom	Button		Mở form thêm phòng mới
6	txtSearch	TextBox		Tìm kiếm phòng theo số phòng
7	cboStatusFilter	ComboBox		Lọc phòng theo trạng thái (Tất cả, Trống, Đã đặt, ...)
8	tblRooms	Table		Bảng danh sách phòng
9	colRoomNumber	Column		Cột số phòng
10	colFloor	Column		Cột tầng
11	colRoomType	Column		Cột loại phòng
12	colStatus	Column (Badge)		Cột trạng thái (Đã đặt, Trống)
13	colNote	Column		Cột ghi chú
14	btnActions	Button (Icon)		Menu thao tác (sửa, xóa) cho từng dòng
15	lblPagination	Label		Hiển thị thông tin phân trang

16	btnPrevPage	Button		Chuyển trang trước
17	btnNextPage	Button		Chuyển trang sau

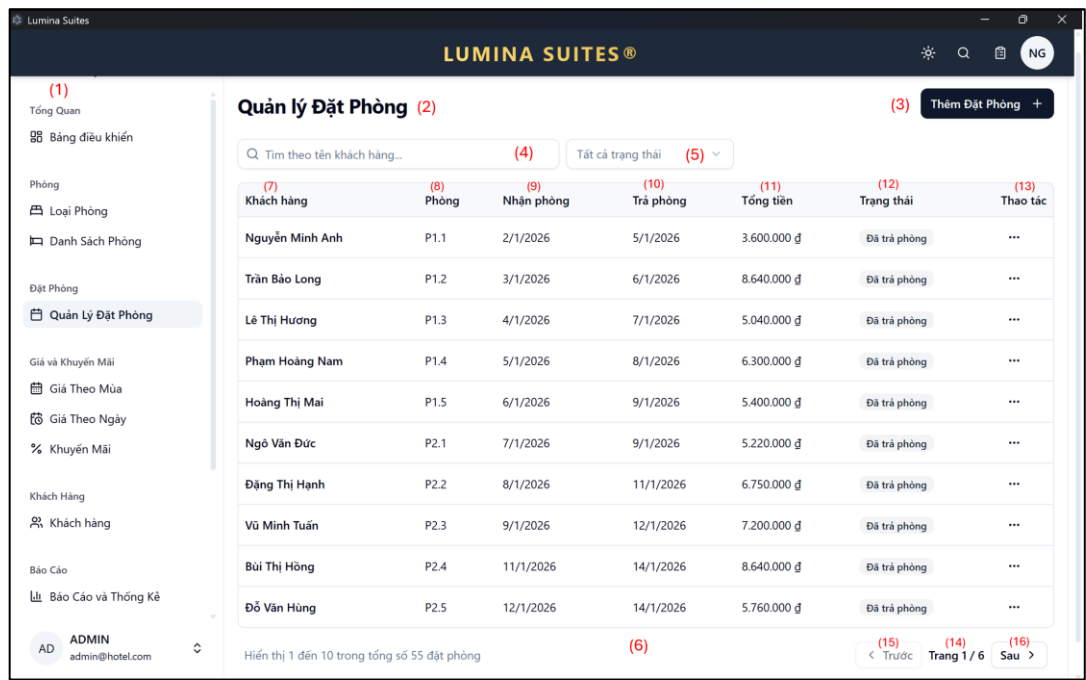
c. Danh sách các biến cố và xử lý tương ứng

Bảng 5.19: Danh sách các biến cố và xử lý tương ứng - Quản lý phòng

STT	Biến cố	Xử lý	Ghi chú
1	btnAddRoom	Mở dialog thêm phòng mới. Nhập số phòng, tầng, loại phòng, ghi chú. Lưu thành công sẽ cập nhật lại bảng.	Gọi API POST /rooms
2	btnBatchAdd	Mở dialog thêm hàng loạt phòng. Cho phép thêm nhiều phòng cùng lúc theo tầng và loại phòng.	
3	txtSearch	Lọc danh sách phòng theo số phòng. Kết quả cập nhật real-time trong bảng	
4	cboStatusFilter	Lọc danh sách phòng theo trạng thái (Tất cả, Trống, Đã đặt, Bảo trì, Đã giữ). Cập nhật bảng ngay khi chọn.	
5	btnActions (Sửa)	Mở dialog chỉnh sửa thông tin phòng được chọn. Cho phép cập nhật trạng thái, ghi chú, loại phòng.	Gọi API PUT /rooms/{id}
6	btnActions (Xóa)	Hiện thị dialog xác nhận xóa phòng. Nếu xác nhận, xóa phòng và cập nhật lại bảng	Gọi API DELETE /rooms/{id}
7	btnPrevPage	Chuyển về trang trước trong danh sách phân trang.	
8	btnNextPage	Chuyển sang trang tiếp theo trong danh sách phân trang.	

5.3.9 Màn hình quản lý đặt phòng

a. Giao diện màn hình



Hình 5.11: Giao diện quản lý đặt phòng

b. Mô tả các đối tượng trên màn hình

Bảng 5.20: Mô tả các đối tượng trên màn hình quản lý đặt phòng

STT	Tên	Kiểu	Ràng buộc	Chức năng
1	navSidebar	NavigationPanel		Menu điều hướng bên trái
2	lblTitle	Label		
3	btnAddBooking	Button		
4	txtSearch	TextBox		
5	cboStatusFilter	ComboBox		
6	tblBookings	Table		
7	colCustomerName	Column		
8	colRoom	Column		
9	colCheckIn	Column		
10	colCheckOut	Column		
11	colTotalPrice	Column		
12	colStatus	Column		

13	btnActions	Button (Icon)		
14	lblPagination	Label		
15	btnPrevPage	Button		
16	btnNextPage	Button		

c. Danh sách các biến cố và xử lý tương ứng

Bảng 5.21: Danh sách các biến cố và xử lý tương ứng - Quản lý đặt phòng